

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Recap 2
Neural Networks
Stochastics
Sensor Modeling

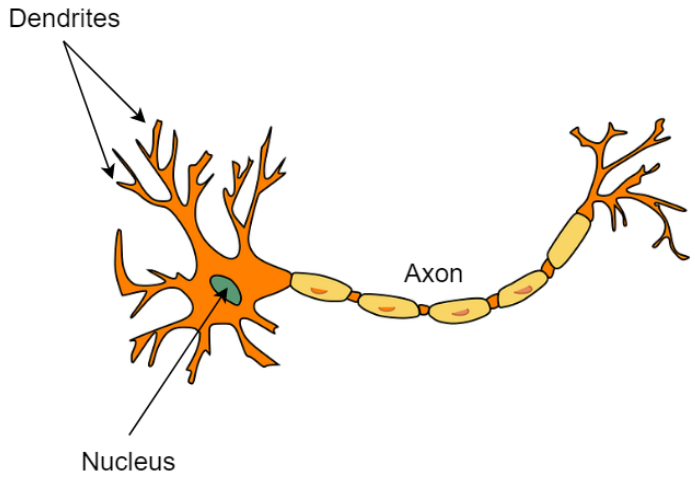
June 17, 2026

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler
Technische Hochschule Ingolstadt
AIMotion, AI and Sensor Data Fusion
Email: bjoern.giesler@thi.de

- **Preparing for the Exam**
  - Use the slides!!!
    I am updating them to have a bit more consistent variable naming etc., but everything in the exam is in the slides, and in the exercises.
  - Use the exercises!
    I will not have time unfortunately to create more exercise sheets.
    Maybe AI can prepare some sheets for you?
  - I will publish a list of literature, Youtube videos etc. that can also help you prepare.
  - Study together, in pairs or groups!
    There's always someone in a group who has good questions, or good understanding.
- These recap slides are *not enough* to prepare for the exam. Use the original slides and the exercises.

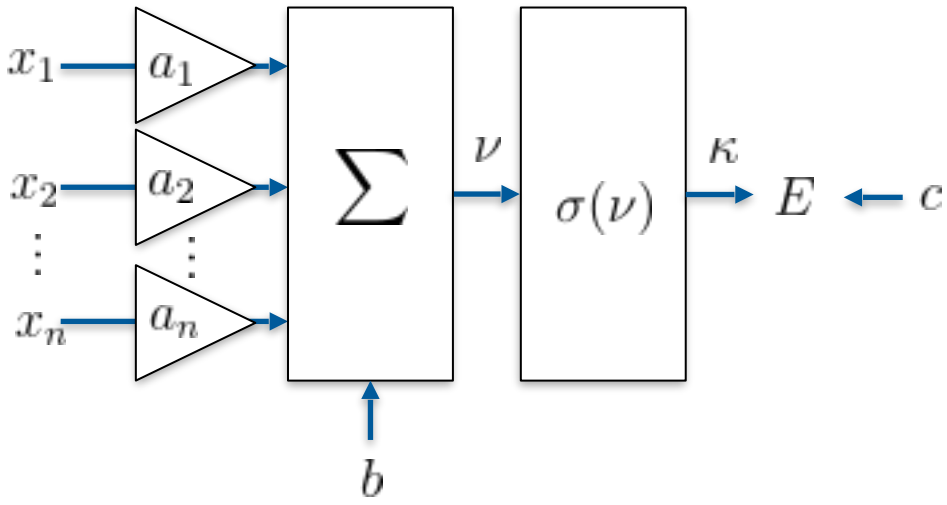
Machine Learning: Neurons Gradient Descent



- **There are many classification algorithms: decision trees, k-nearest neighbor clustering, support vector machines, ...**
- **What is commonly called AI is built on the Artificial Neuron.**
- **It is modelled after a biological neuron, which**
  - collects electrical impulses in its dendrites
  - produces an output in its nucleus and forwards it through its axon,
  - which connects to other neurons‘ dendrites.
- **The AN produces the output**

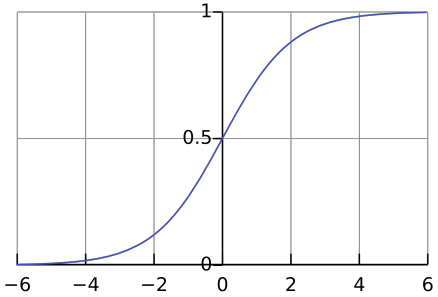
$$\kappa = \sigma(\sum_i x_i a_i + b) = \sigma(\mathbf{x}\mathbf{a}^\top + b)$$

- **In fixed networks, the weights and biases a_i, b form the *model.***
- **σ is the activation function, with several classical options.**



- **One of the most common activation functions is the *sigmoid function***

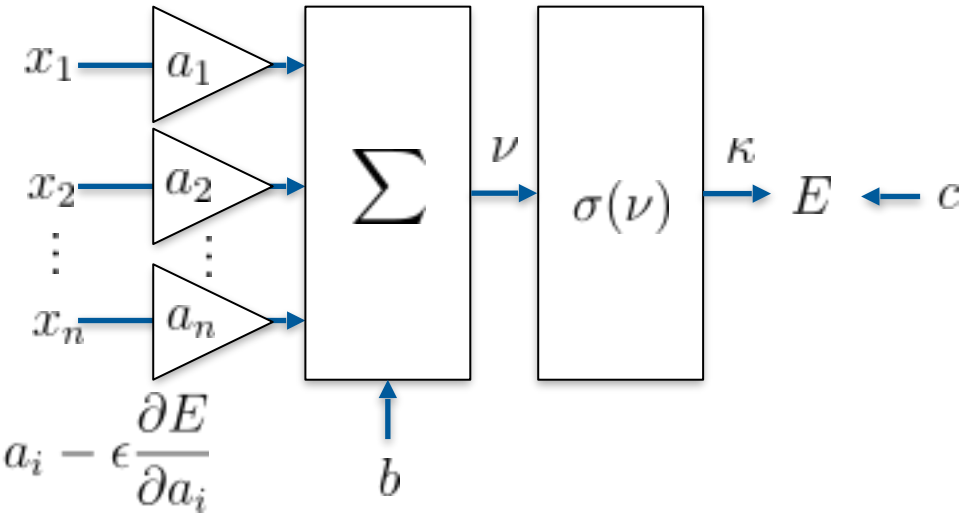
$$\sigma(\nu) = \frac{1}{1 + e^{-\nu}}$$



  It is continuous and differentiable, approaches a binarization function for sufficiently large input values, and its output can be easily interpreted by developers.

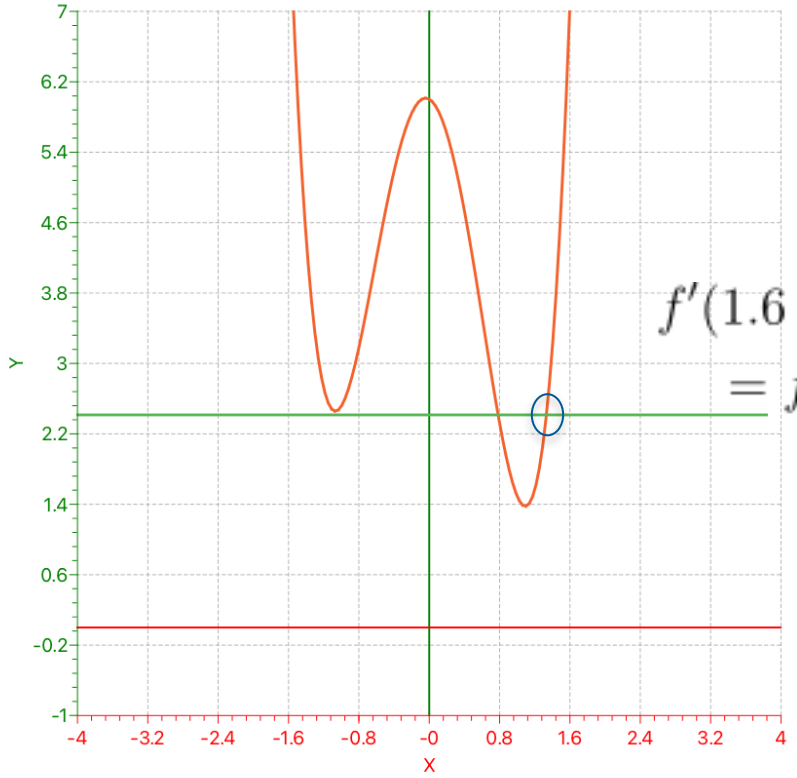
- **If we want to train a single neuron, we need to adapt its weights according to the error of its output. We can calculate the absolute error as**

$$E(\mathbf{a}) = \frac{1}{2}(c - \kappa(\mathbf{a}))^2 = \frac{1}{2}(c - \sigma(\mathbf{x}\mathbf{a}^\top + b))^2$$



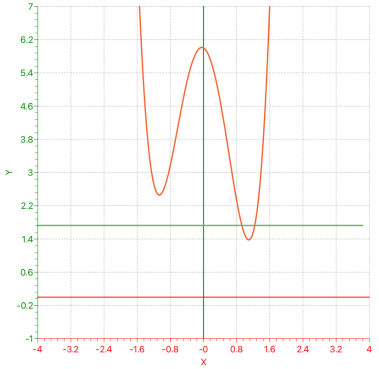
- **To break this down to the individual weights, we calculate the partial derivatives $\frac{\partial E}{\partial a_i}$ and apply them to the weights incrementally with a scaling factor ϵ: $a_i' = a_i - \epsilon \frac{\partial E}{\partial a_i}$**

- **The scaling factor is usually very small to *descend along the gradient* in the right direction.**

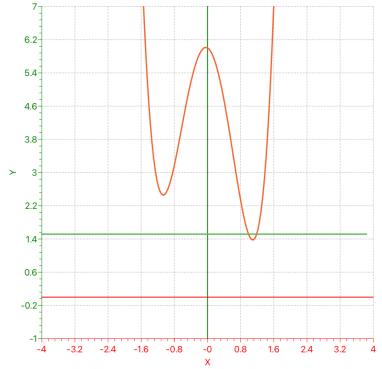


$$f'(1.6 - 26.25\epsilon) = f'(1.3375) \approx 9.5$$

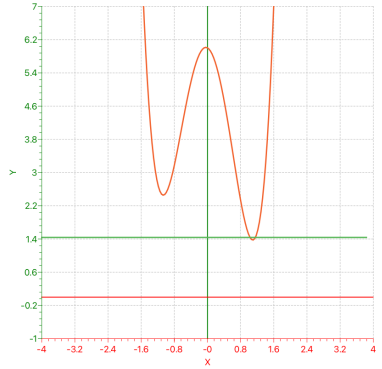
$$f'(1.3375 - 9.5\epsilon) = f'(1.2425) \approx 5.12$$



$$f'(1.1425 - 5.12\epsilon) = f'(1.1913) \approx 3.11$$



$$f'(1.1425 - 3.11\epsilon) = f'(1.1602) \approx 2$$



$$f'(1.14) \approx 1.32$$
$$f'(1.127) \approx 0.99$$
$$f'(1.117) \approx 0.58$$
$$f'(1.111) \approx 0.4$$
$$\vdots$$
$$f'(1.09756) \approx 0$$
$$f(1.09756) \approx 1.3722$$

We arrive at a local minimum of 1.3722 at position x=1.09756.

For the multi-variable case $f(x_1, x_2, \ldots, x_n)$**,**

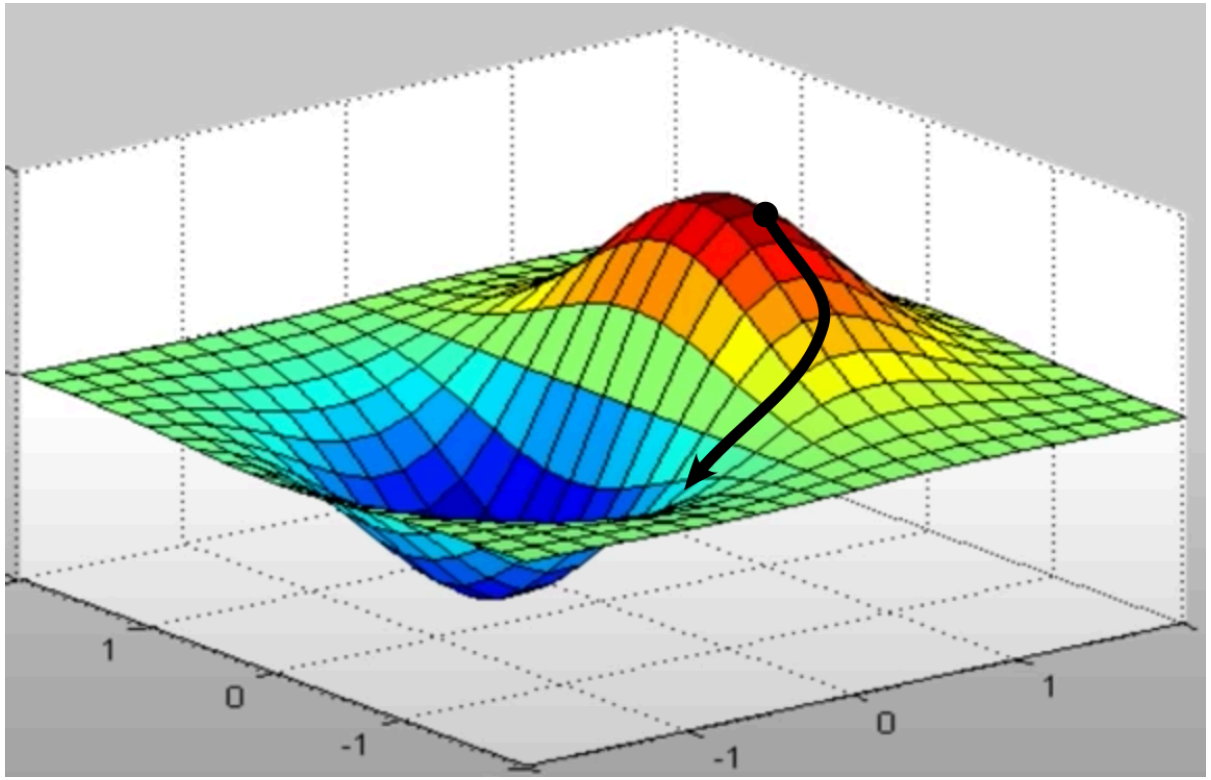
we need the *gradient vector,* i.e. the vector of partial derivatives

$$f'(x_1, x_2, \ldots, x_n) = \nabla f = \begin{pmatrix} \partial f/\partial x_1 \\ \partial f/\partial x_2 \\ \vdots \\ \partial f/\partial x_n \end{pmatrix}$$

We start at a random point in *n*-dimensional space and calculate ∇f **.**

We need to *descend in the direction of* $-\nabla f$**.**

The *length of* $-\nabla f$ **gives us an indication of how much we need to descend.**

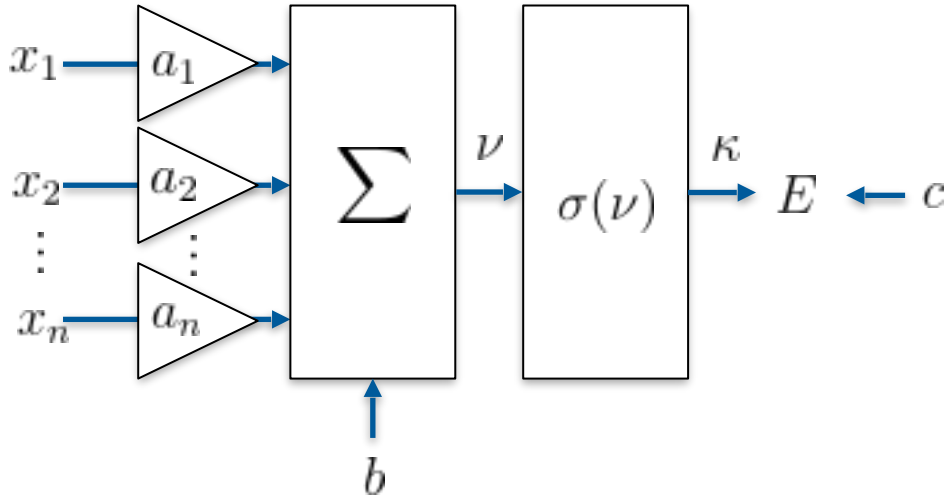


- **Let‘s look at a single neuron.**
- **We want to find the *minimum of the error function***

$$E(\mathbf{a}) = \frac{1}{2}(c - \kappa(\mathbf{a}))^2 = \frac{1}{2}(c - \sigma(\mathbf{x}\mathbf{a}^\top + b))^2$$

using gradient descent. So we need the gradient vector

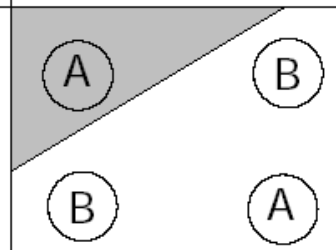
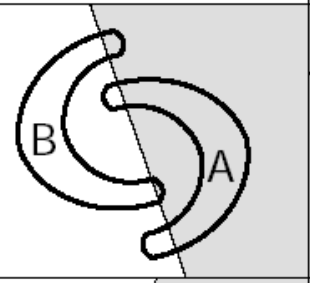
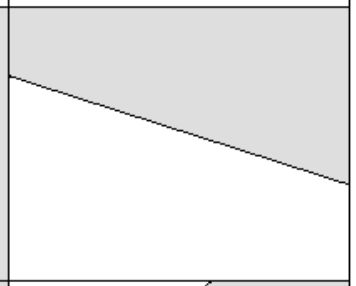
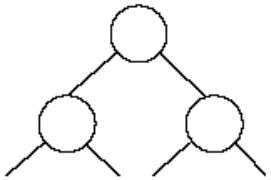
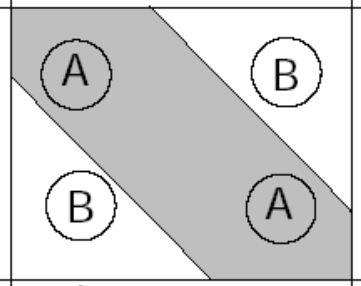
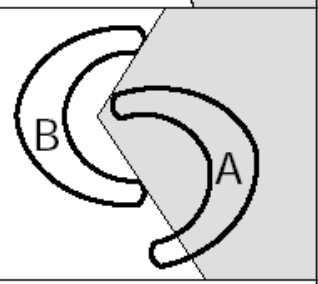
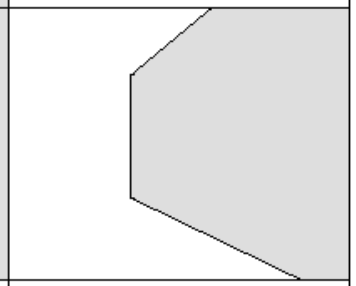
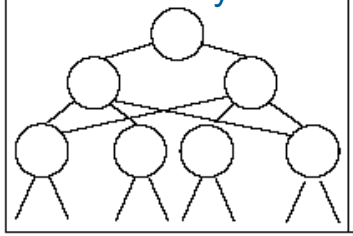
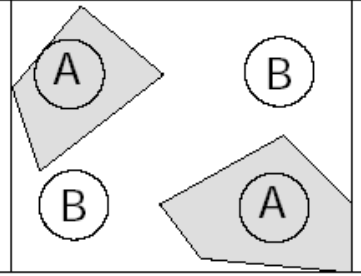
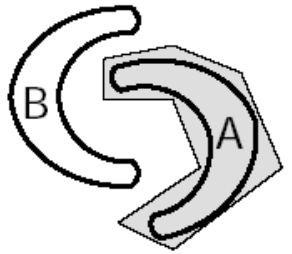
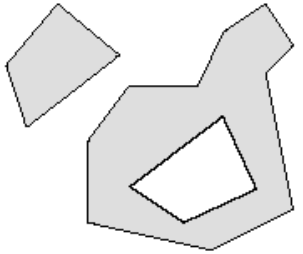
$$\nabla E = \begin{pmatrix} \partial E/\partial a_1 \\ \partial E/\partial a_2 \\ \vdots \\ \partial E/\partial a_n \end{pmatrix}$$



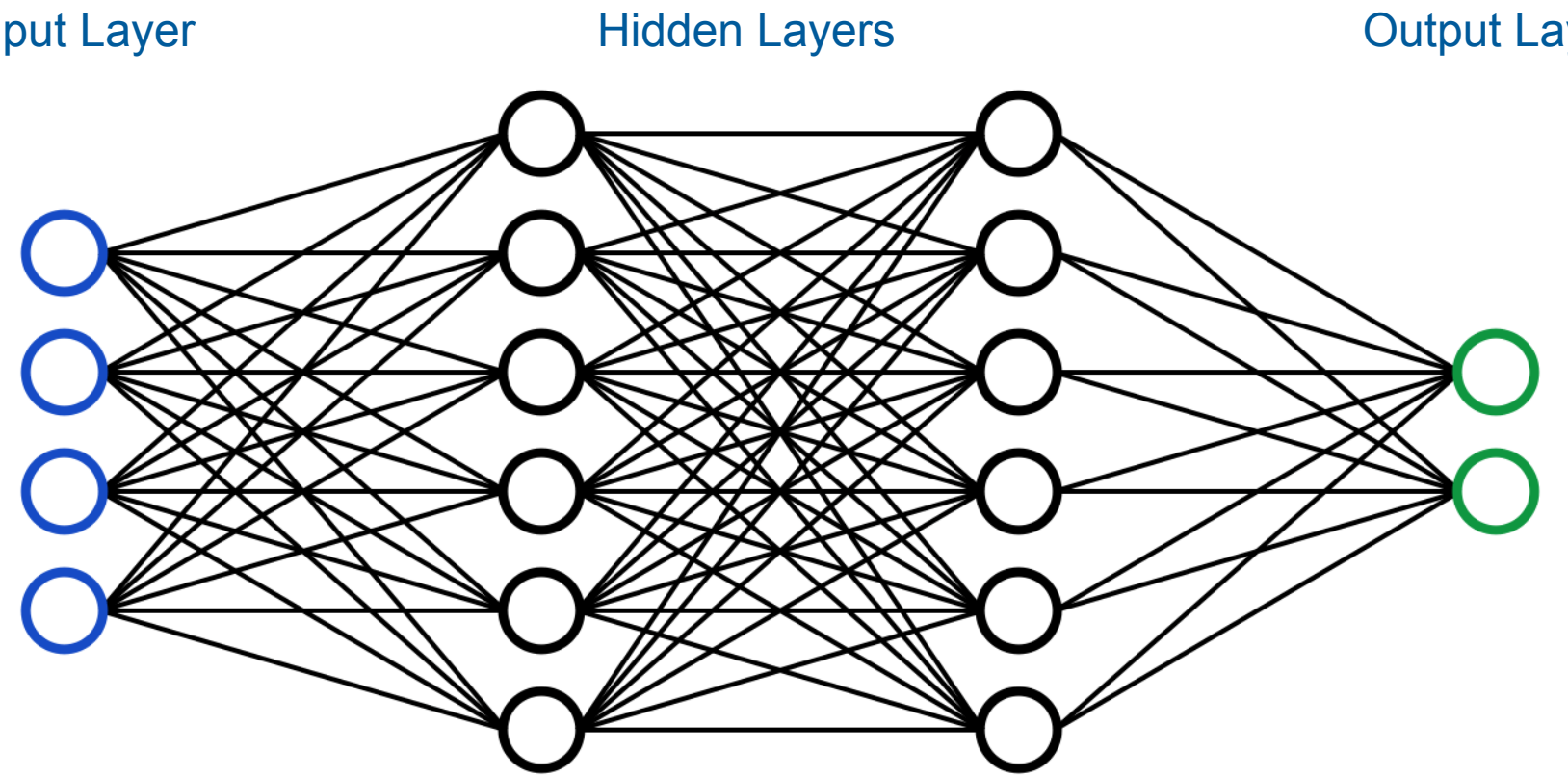
- **Applying the chain rule twice, the individual components of the gradient vector are**

$$\frac{\partial E}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial \nu}\frac{\partial \nu}{\partial a_i}$$

- **A single neuron models a single line (or hyperplane) separator. It can for example not model XOR.**
- **Like in a human brain, we construct multiple layers of neurons for more general separators.**

Network Structure	Separation Region	XOR separation	Nested Region Separation	General Region Separation
One layer 	Single plane			
Two layers 	Convex regions			
Three layers 	Arbitrarily complex regions			

For more difficult problems, we build up multi-layer networks of many neurons.



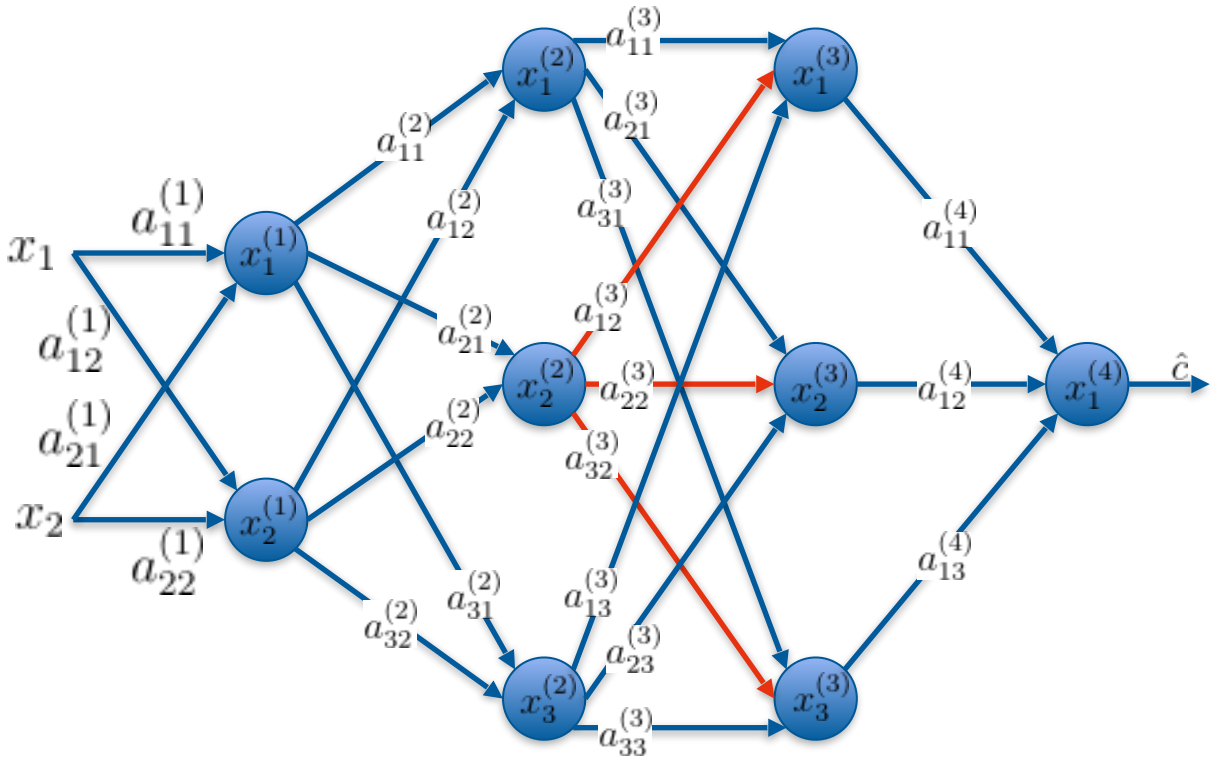
- **Let‘s look at a network with a bit more complex topology.**
- **The derivation for the nodes in the output layer remains the same –**

  the weights $a_{jk}^{(L)}$ are only influenced by $\hat{c} = x_i^{(L)}$.
- **To backpropagate, we need to consider that the local error is caused by influences coming down multiple paths:**

$$\frac{\partial E}{\partial a_{jk}^{(L)}} = \frac{\partial E}{\partial x_j^{(L)}} \frac{\partial x_j^{(L)}}{\partial \nu_j^{(L)}} \frac{\partial \nu_j^{(L)}}{\partial a_{jk}^{(L)}}$$

becomes

$$\frac{\partial E}{\partial x_k^{(L-1)}} = \sum_{j=0}^{n_{L-1}} \frac{\partial E}{\partial x_j^{(L)}} \frac{\partial x_j^{(L)}}{\partial \nu_j^{(L)}} \frac{\partial \nu_j^{(L)}}{\partial x_k^{(L-1)}}$$



- **For each example m in the training set, we can calculate an error E_m by forward-calculating the network output and subtracting it from the output pattern the network is supposed to approximate.**
- **We are interested in the performance of the network over the full training set, which we can average out:**

$$E_{\text{avg}} = \frac{1}{n}\sum_{m=1}^{n} E_m$$

$$\frac{\partial E_{\text{avg}}}{\partial x^{(L)}} = \frac{1}{n}\sum_{m=0}^{n} \frac{\partial E_m}{\partial x^{(L)}}$$

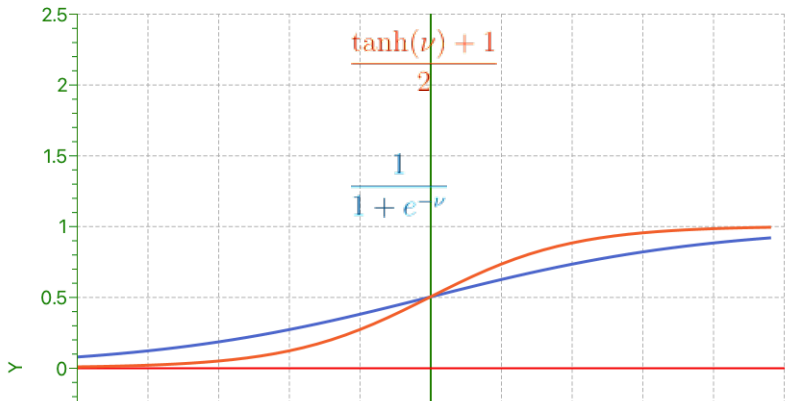
- **I.e. our gradient vector is computed as the average of the gradient vectors of the individual training examples.**

To conclude,

1. We initialize the network with random weights and biases, obtaining the vector
$$N = (a_{11}^{(1)}, a_{21}^{(1)}, \ldots, b_1, a_{12}^{(1)}, a_{22}^{(1)}, \ldots, b_2, \ldots, a_{jk}^{(N)}, b_k)^\top$$
This vector, together with the activation functions $\sigma_k^{(N)}$, describes our network.
2. We forward-compute the output error for each output neuron throughout the network by subtracting it from the setpoint output given by the training set.
3. We obtain the average error E_{avg} for all examples in the training set.
4. We compute the vector of partial derivatives as a function of the weights, biases, and average output error,
$$\nabla E = (\frac{\partial E}{\partial a_{11}^{(1)}}, \frac{\partial E}{\partial a_{21}^{(1)}}, \ldots, \frac{\partial E}{\partial b_1}, \frac{\partial E}{\partial a_{12}^{(1)}}, \frac{\partial E}{\partial a_{22}^{(1)}}, \ldots, \frac{\partial E}{\partial b_2}, \ldots, \frac{\partial E}{\partial a_{jk}^{(N)}}, \frac{\partial E}{\partial b_k})^\top$$
5. We descend the gradient given by ∇E „an ϵ-sized step“ by modifying the network:
$$N' = N - \epsilon \nabla E$$

Activation Functions

Sigmoid: $\sigma(\nu) = \frac{1}{1+e^{-\nu}}$ $\quad \frac{\partial \sigma}{\partial \nu} = \sigma(\nu)(1-\sigma(\nu))$

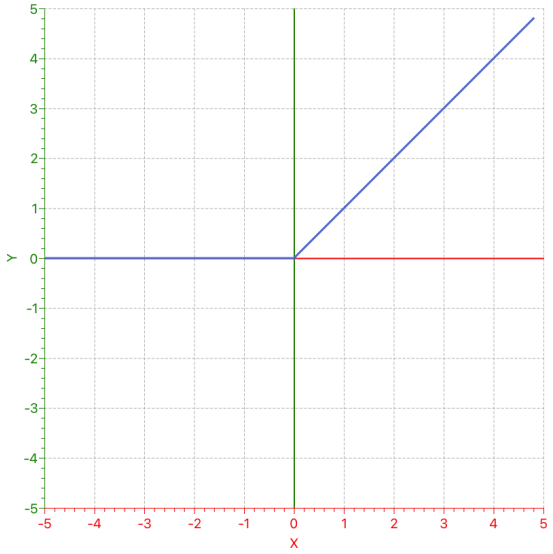


Tangens hyperbolicus:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\frac{\partial}{\partial dx}\frac{e^x - e^{-x}}{e^x + e^{-x}} = 1 - \left(\frac{e^x - e^{-x}}{e^x + e^{-x}}\right)^2 = 1 - \tanh(x)^2$$

ReLU: $\mathrm{ReLU}(x) = max(0,x) = \frac{x+|x|}{2}; \frac{\partial \mathrm{ReLU}}{\partial x} = \begin{cases} 1 & : x \geq 0 \\ 0 & : \text{otherwise} \end{cases}$



Measuring Quality

Precision and Recall: Classification Quality

- ***Precision*** measures *how many classifications are correct*:

$$\text{Precision} = \frac{\#\text{True Positive Classifications}}{\#\text{Total Classifications}} = \frac{tp}{tp + fp}$$

- ***Recall*** measures *how many instances were identified:*

$$\text{Recall} = \frac{\#\text{True Positive Classifications}}{\#\text{Total Classifiable Objects}} = \frac{tp}{tp + fn}$$

- Example:
  - An image contains 10 cars.
  - 7 were correctly identified.
  - 2 non-car patterns were classified as cars (False Positives).
  - 3 cars were not identified (False Negatives).

Instance Is Detected \ Instance Exists	Yes	No
Yes	True Positive (TP)	False Positive (FP)
No	False Negative (FN)	True Negative (TN)

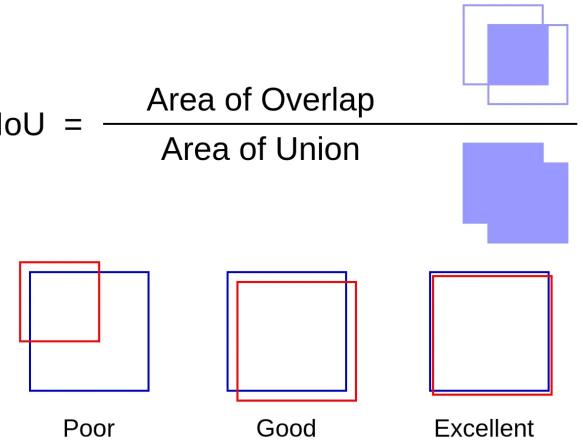
$$\text{Precision} : (7)/(7+2) = 77.8\%$$
$$\text{Recall} : 7/10 = 70\%$$

Measuring Quality

- ***F1 Score*: Classification Quality**
  - The F1 score is the *harmonic mean* of Precision and Recall: $F = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
  - It unifies Precision and Recall into one „goodness score“.

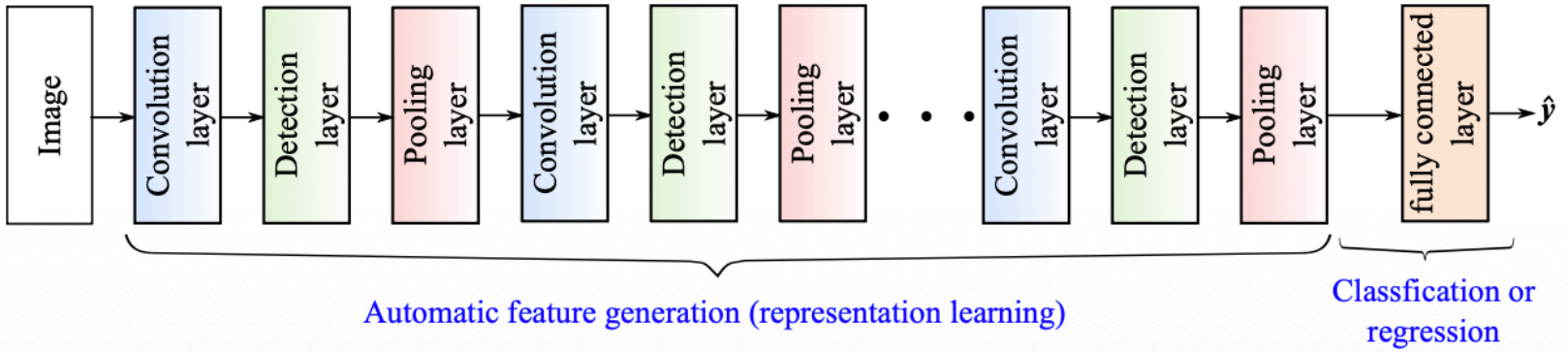
- **Intersection Over Union: Spatial Localization Quality**
  - Measures the normalized overlap of two bounding boxes (e.g. ground truth and prediction).
  - Gives a good estimate on precision of the bounding box extraction, vital for object **spatial localization** .

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



Machine Learning: Convolutional Neural Networks

- **Convolutional Neural Networks (ger. Faltungsnetzwerke) are well suited for processing time series or image data, as they are able to find local features in the data and learn them.**
- **CNNs are very similar to multilayer perceptrons, but they require less parameters for the same number of hidden layers.**
- **This is realized by replacing at least one weighting operation $\mathbf{a}^\top \mathbf{x} + b$ by another linear operation: the Convolution or the Deterministic Cross-Correlation, respectively.**



- **Deep CNNs consist of a sequence of convolution, detection and pooling layers, completed by a fully connected layer.**

- **We can think of an image as a 2D function, $\mathrm{Img}(u, v) \rightarrow [0, I_{max}]$, where u, v are the image coordinates and Imax is the maximum intensity allowed by the image.**
- **Object characteristics take the form of local image changes in one or two directions.**
- **We can amplify these changes for later processing by deriving the image in both directions.**
- **The derivative again takes the form of an image.**





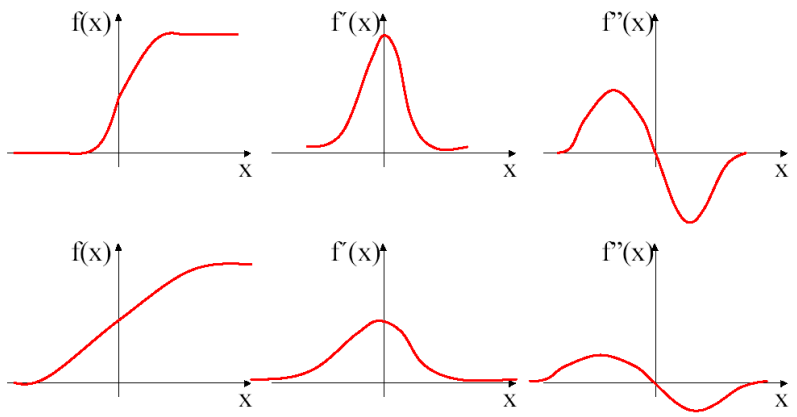
Low change



High change in two directions



High change in one direction



- **The convolution of two discrete D-dimensional signals $v(n_1, n_2, ..., n_D)$ and $w(n_1, n_2, ..., n_D)$ can be written as**

$$x(n_1, n_2, ..., n_D) = v(n_1, n_2, ..., n_D) *\overset{D}{\cdots}* w(n_1, n_2, ..., n_D)$$

$$= \sum_{k_1=-\infty}^{\infty} \sum_{k_2=-\infty}^{\infty} \cdots \sum_{k_D=-\infty}^{\infty} v(n_1 + k_1, n_2 + k_2, \ldots, n_D + k_D) w(k_1, k_2, \ldots, k_D)$$

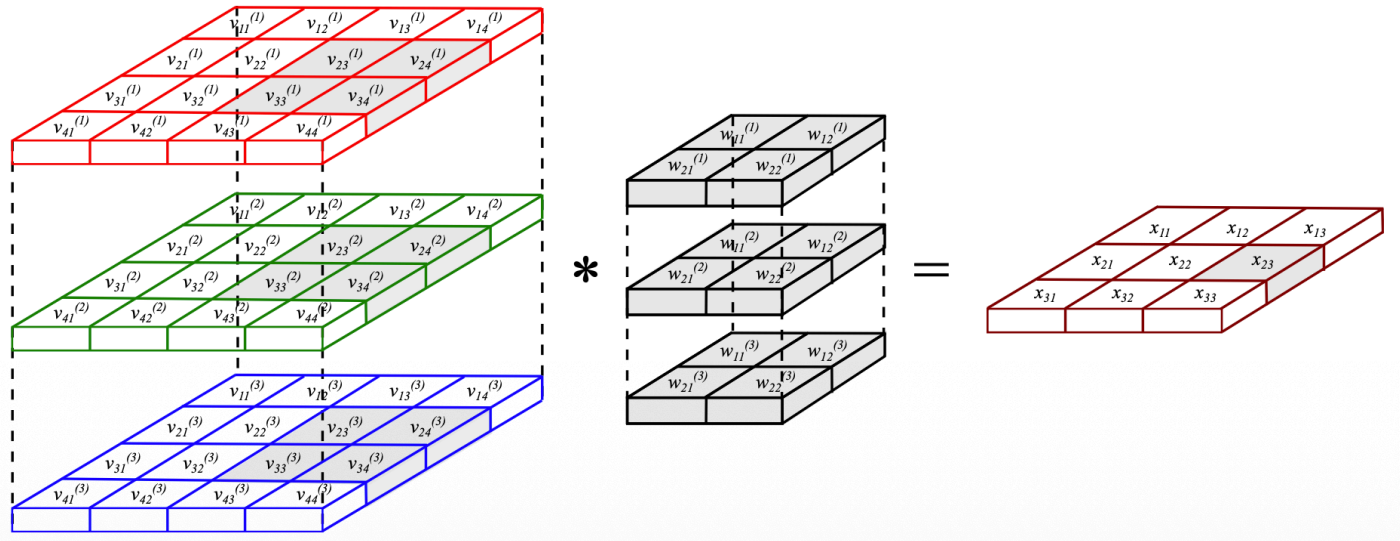
  (technically, that is the cross-correlation, convolution would be $v(n_1 - k_1, n_2 - k_2, \ldots)$, but in computer vision these are typically used interchangeably as they differ only by sign).

- **For D=2, the data structure can be interpreted as an image.**
- **We have seen convolution kernels in lecture 3.**

$$\frac{1}{16}\begin{vmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{vmatrix} \qquad C_{\mathrm{Sx}} = \begin{vmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{vmatrix}, C_{\mathrm{Sy}} = \begin{vmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{vmatrix}$$

- **In CNNs, the kernel is *learned and not predetermined*.**

- **The advantage of the convolution operation is the capturing of local structure in an image, thereby learning the typical required patterns.**
- **In lecture 3 we have discussed that convolution is typically done on a grayscale image after converting RGB to grayscale.**
- **However we can think of RGB images as 3-dimensional data structures with the more general convolution equation.**



Convolution of 4x4x3 image with 2x2x3 kernel and adding up to a 3x3x1 image

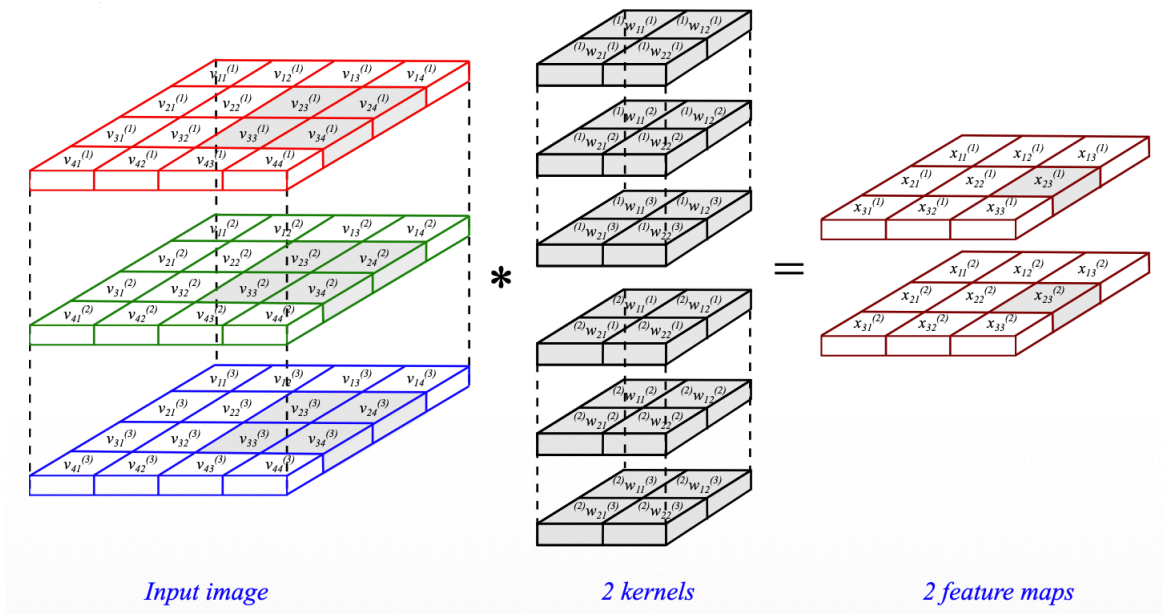
The values $x_{11}, \ldots, x_{33}$ are derived from the convolution of the 4x4x3 image with the 2x2x3 kernel:

$$\begin{aligned} x_{23} = & v_{23}^{(1)} w_{11}^{(1)} + v_{24}^{(1)} w_{12}^{(1)} + v_{33}^{(1)} w_{21}^{(1)} + v_{34}^{(1)} w_{22}^{(1)} \\ & + v_{23}^{(2)} w_{11}^{(2)} + v_{24}^{(2)} w_{12}^{(2)} + v_{33}^{(2)} w_{21}^{(2)} + v_{34}^{(2)} w_{22}^{(2)} \\ & + v_{23}^{(3)} w_{11}^{(3)} + v_{24}^{(3)} w_{12}^{(3)} + v_{33}^{(3)} w_{21}^{(3)} + v_{34}^{(3)} w_{22}^{(3)} \end{aligned}$$

where the high index denotes the R, G, and B channel.

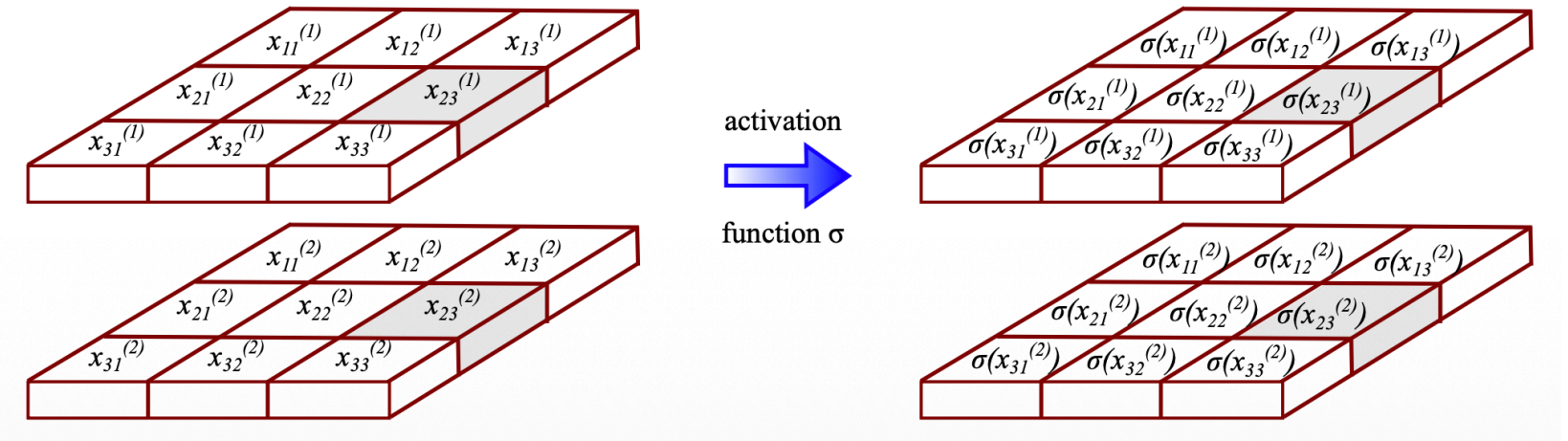
- **We can interpret the w_{ij} as the input weights to a neuron.**
- **In order to dertermine the 9 values $x_{11}, \ldots, x_{33}$ we only need to learn 3*4+1= 13 parameters.**
- **Solving the same task with a perceptron requires 16*3*9+9= 441 parameters.**

- **Usually we do not just generate one feature map from an image, but several.**
- **These are obtained by convolving with different kernels K_F .**

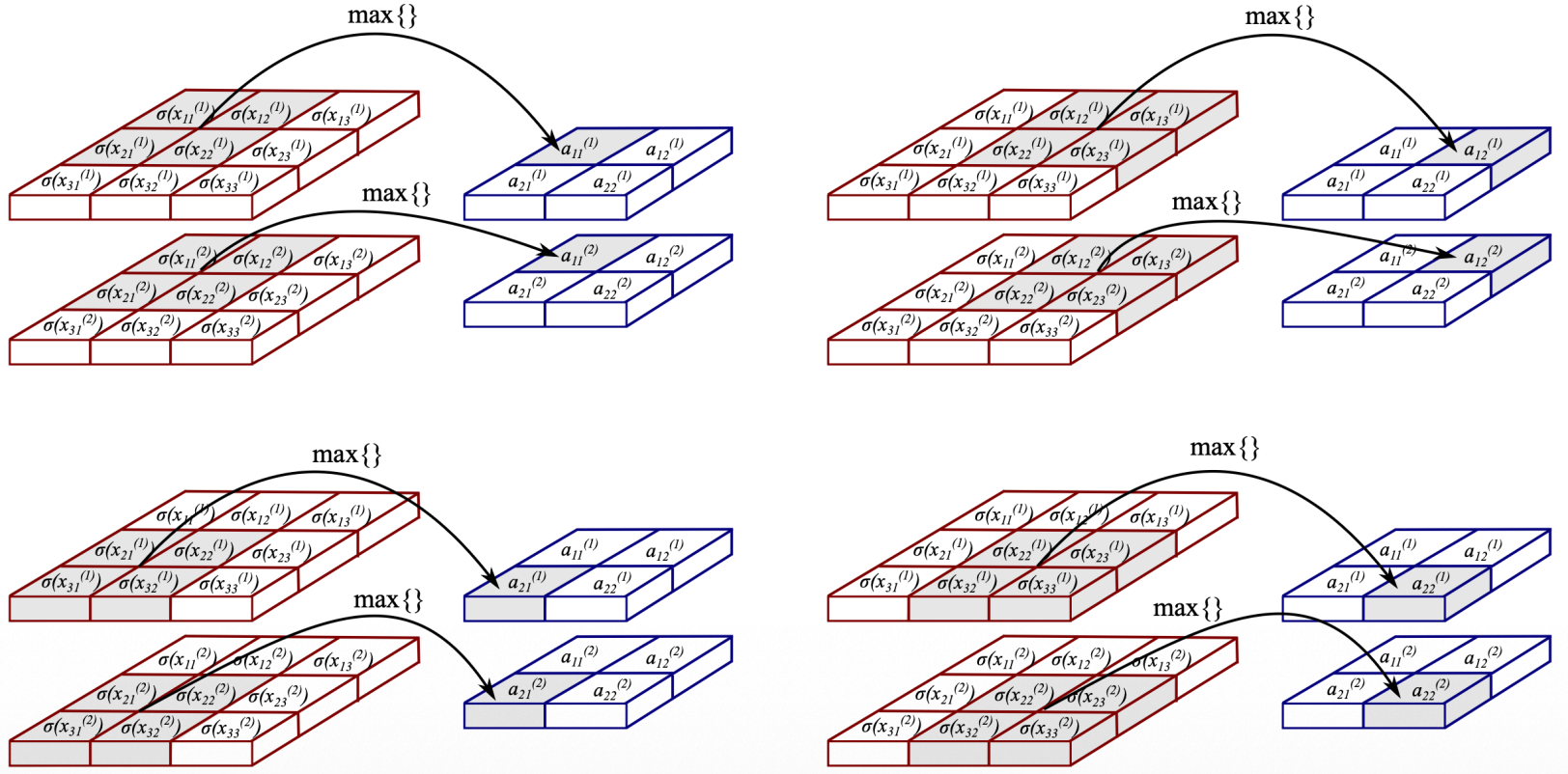


- **So far, we are in the convolution layer of a CNN, where everything is strictly linear.**

- **Detection Layers introduce nonlinearities to CNNs and are realized with activation functions, as previously discussed in the multilayer perceptron section.**
- **The activation functions affect the feature maps element-wise.**
- **Often the ReLu function, introduced in Subsection 2.1, is used as an activation function.**
- **The dimensions of the input and output keep the same size after the detection layer.**

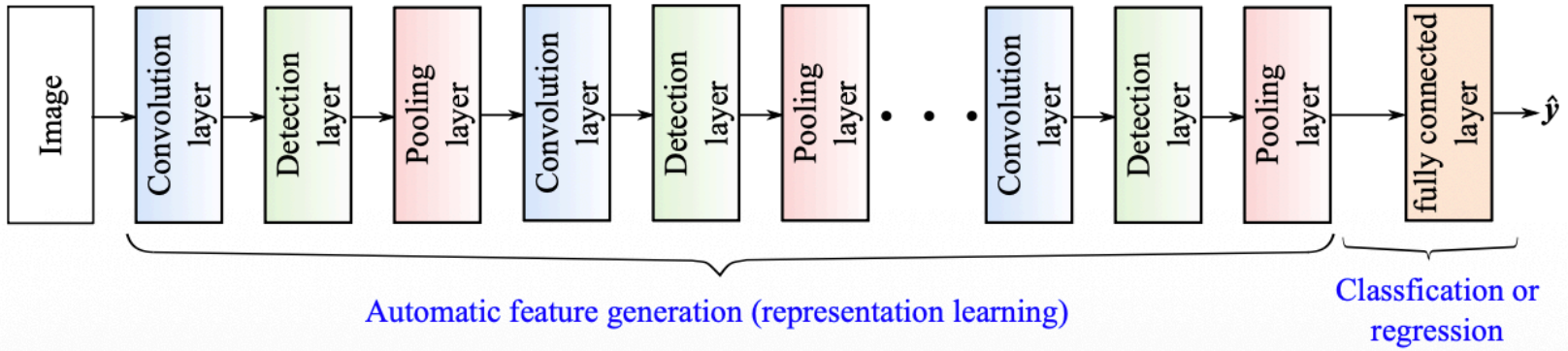


- **Pooling Layers, often also referred to as downsampling layers, reduce the resolution of the feature map by replacing areas of the feature map with statistical quantities.**
- **Pooling yields an invariance with regard to minor translations of the input, i. e., small local translations in the input data structure lead to no effect at the output of the pooling layer.**
- **Often „Max Pooling“ is used – only the maximum value from a section is kept and all the other values are discarded.**
- **Usually a quadratic region with length and width L_P is chosen, which is slid over the feature map with a defined step size (also named stride) S_P and the covered area is replaced by one statistical quantity, e. g., the maximum value.**

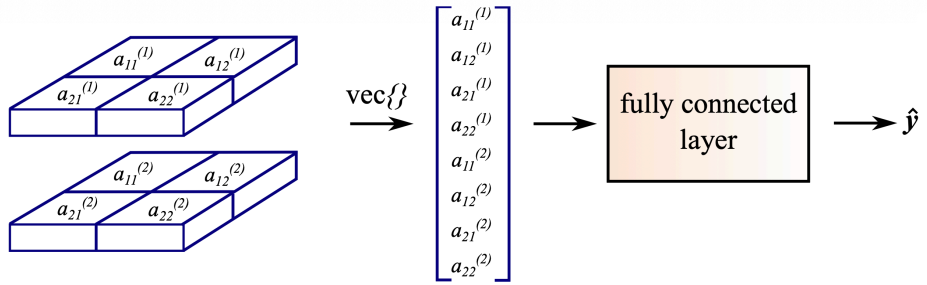


Max Pooling with $L_P = 2, S_P = 1$

- **A CNN usually is set up with several convolution, detection and pooling layers, finally followed by a fully connected layer.**



- **The fully connected layer transforms the final feature maps to the output of the network.**
- **Therefore the feature maps are rearranged to a single vector using the vec operator.**
- **This vector is then the input for the fully connected layer, which computes the output of the CNN.**



- **Backpropagation follows the same general rule for multilayer perceptrons, with the following specialties:**
  - The activation functions in the fully connected layer(s) and the detection layers are differentiated as usual.
  - The convolution layer does not have an activation function; its derivative is simply the image at the given position:

$$\frac{\partial x_{ij}}{\partial w_{kl}} = \frac{\partial}{\partial w_{kl}} \sum_m \sum_n v_{i+m,j+n} w_{mn} = v_{i+k,j+l}$$

  - The statistical function used in the pooling layers may not be differentiable. The maximum function for example is not. We can use a convention (a „subdifferential“):

$$\frac{\partial \max(p,q)}{\partial p} = \begin{cases} 1 & \text{if } p > q \\ 0 & \text{else} \end{cases}$$

Stochastics

Example: Random Experiment

- Throwing a fair 6-sided die: Six possible outcomes ω_i from sample space

$$\Omega = \{"1", "2", "3", "4", "5", "6"\}$$

Random Variable / Realization of a Random Variable:

- A **function** $\mathrm{X}(\omega)$, which assigns a real number x to every outcome $\omega \in \Omega$, is called **random variable**, so we have

$$x = X(\omega)$$

- The real number x for one specific outcome is denoted as **realization of a random variable** $\mathrm{X}(\omega)$,

Example 1:

- Throwing a dice: Possible outcomes ω_i

$$\Omega = \{"1", "2", "3", "4", "5", "6"\}$$

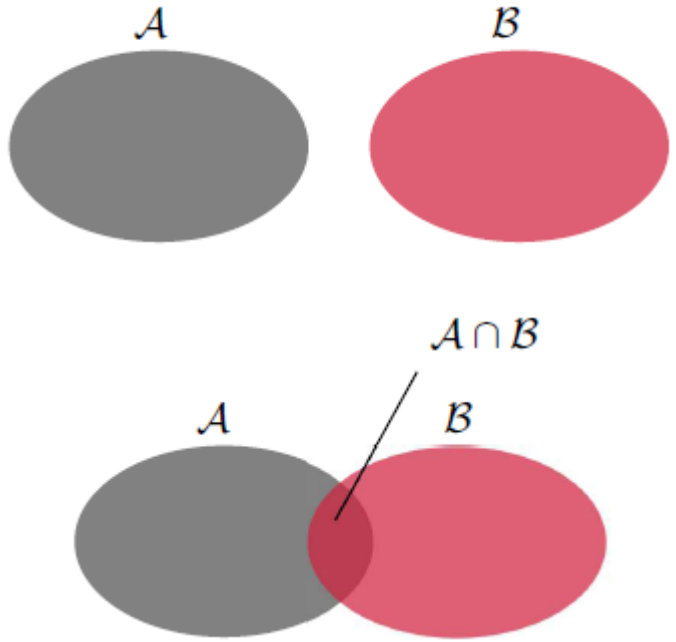
- Number of points on the dice (realization) $x_i = X(\omega_i)$

Example 2:

- Throwing a die : Outcomes ω_i are heads and tails
- Assign a real number to every outcome, i. e. head $\rightarrow 0$, tail $\rightarrow 1$, i.e. $X(\omega) \in \{0, 1\}$

Events

- Sample Set
  $$\Omega = \{\omega_1, \omega_2, \ldots, \omega_L\}$$
- Define an Event $A = \{a_1, a_2, \ldots, a_M\} \subseteq \Omega$ with $M \leq L$
- Certain event $A = \Omega$
- Impossible event $A = \varnothing$
- Complementary event A^c
- Disjoint events: $A \cap B = \varnothing$ (event A and B can not occur jointly)



Disjont events (top) and non-disjoints events (bottom)

Frequency of Events and Probability

- Frequency $h(A)$ and probability $P(A)$

$$P(A) = \lim_{n\to\infty} h_n(A) \quad \text{with} \quad h_n(A) = \frac{\text{number of experiments with outcome A}}{n}$$

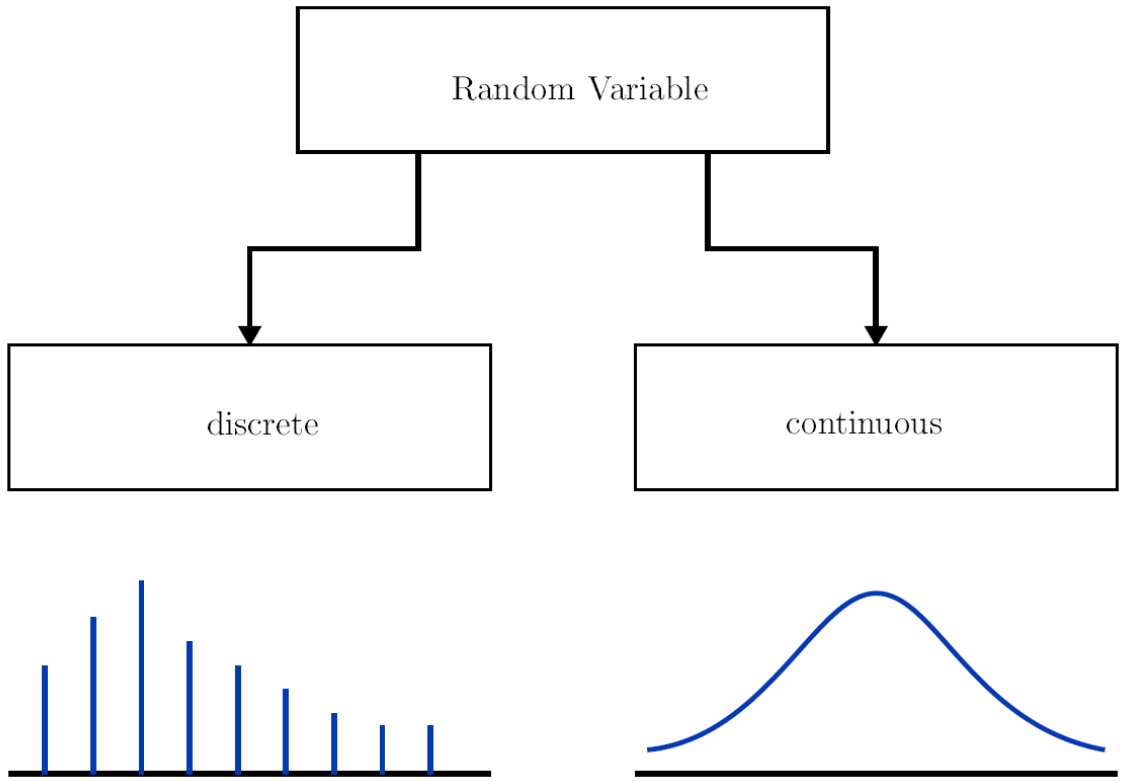
- **Axioms of Kolmogorov**
  - The probability for an event with an outcome between 0 and 1, i. e. it holds $0 \leq P(A) \leq 1$
  - The "certain event" (that is certain to occur) has $P(\Omega) = 1$
  - For disjoint events $A \cap B = \varnothing$ we have that $P(A \cup B) = P(A) + P(B)$

Conclusions:

- Probability of a complementary events A^c is $P\left(A^c\right) = 1 - P(A)$
- Probability of the impossible event is $P(\varnothing) = 0$
- Union of non-disjoint events $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

Discrete vs. Continuous random Variables

- Discrete random variable: Only discrete (throwing dice / coins)
  - Discrete probability density function
- Continuous random variable: Realization can take arbitrary values (Analog output voltage of a sensor)
  - Continuous probability density function



Probability density functions of discrete and continuous random variables

Probability Density Function (PDF)

- Definition (for continuous random variables X) – compare Riemann integrals!

$$p_X(x) = \lim_{dx \to 0} \frac{P(x - dx < x \leq x + dx)}{dx}$$

- Therefore, probability that the realization x takes on values between a and b is computed by integration:

$$P(a < x \leq b) = \int_a^b p_X(x) dx$$

- The probability of the certain event (i.e. anything at all occurring) is always 1

$$P(x \leq \infty) = \int_{-\infty}^{\infty} p_X(x) dx = 1$$

- Note that the probability of the **complementary event** $P(x = C) = 0$
- Please also note the relation to the **(cumulative) distribution function**

$$F_X(x) = P(X \leq x) = \int_{-\infty}^{x} p_X(x) dx$$

Preliminary Remarks:

- Continuous space of possible events (before: discrete)
- Mathematically, discrete events can be described in terms of a continuous probability density function by means of the Dirac-distribution:

$$\delta(x) = \begin{cases} +\infty & \text{if} \quad x = 0 \\ 0 & \text{otherwise} \end{cases}$$

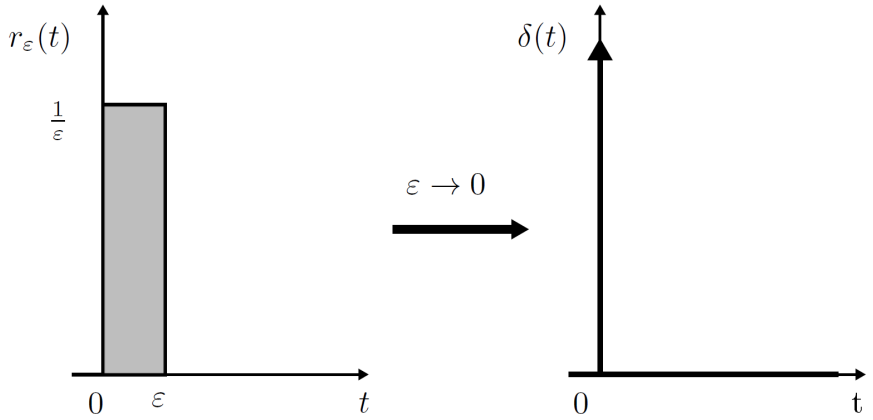
- Gating-characteristic: Due to

$$\int_{-\infty}^{\infty} \delta(t)dt = 1,$$

we have that

$$\int_{-\infty}^{\infty} f(t)\delta(t - t_0)dt = f(t_0)$$

Definition of the Dirac-Function



- Rectangle-shaped function:

$$r_\epsilon(t) = \begin{cases} \frac{1}{\epsilon} & \text{if} \quad 0 \leq t \leq \epsilon \\ 0 & \text{otherwise} \end{cases}$$

- Limiting operation:

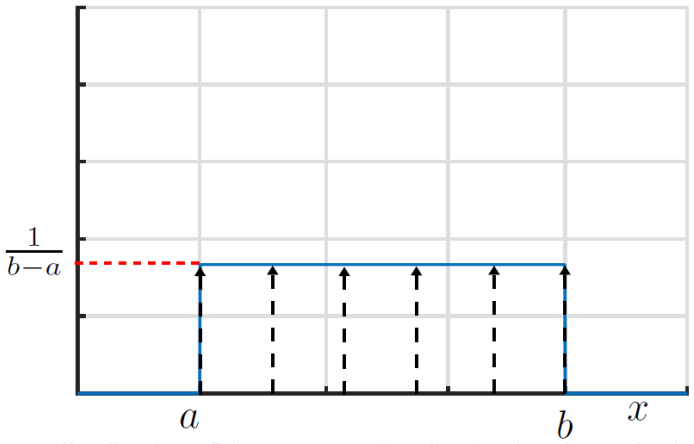
$$\delta(t) = \lim_{\varepsilon \to 0} r_\varepsilon(t)$$

Uniform Distribution:

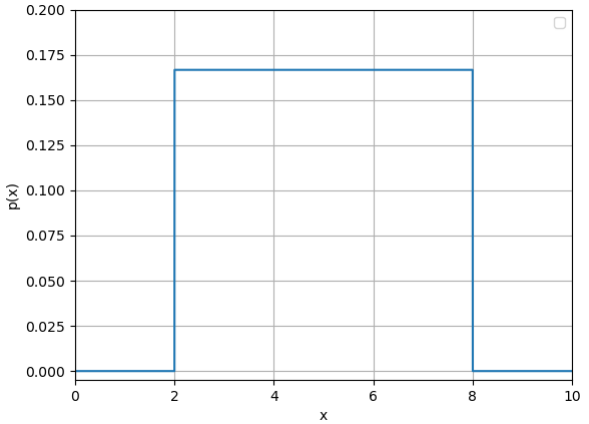
- The probability distribution of a uniformly distributed random variable is

$$p_X(x) = \begin{cases} \frac{1}{b-a} & \text{if} \quad a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

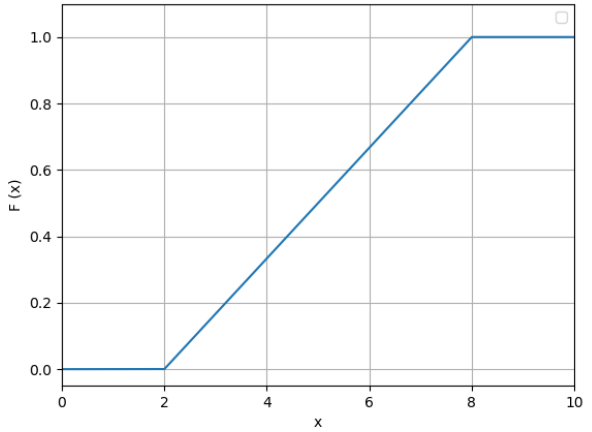
- The discrete counterpart can be represented by means of a convolution with the Dirac distribution (e. g. throwing a dice)



Uniform distribution: Discrete version is obtained employing the Dirac-Impulse



Probability density function



(Cumulative) distribution function

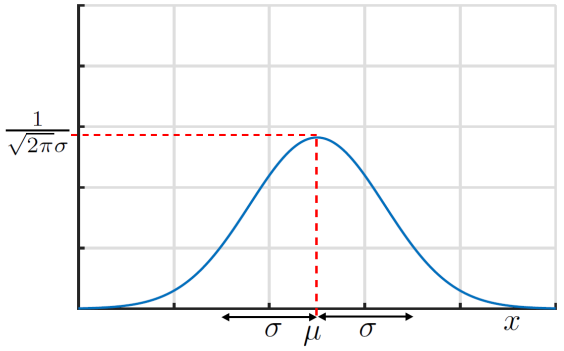
Gaussian-Distribution (Normal Distribution $X \sim N(\mu, \sigma^2)$):

- The probability density function reads:

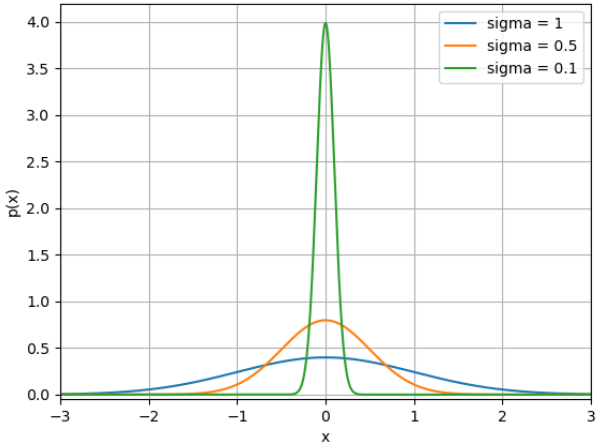
$$p_X(x) = \frac{1}{\sqrt{2\pi} \cdot \sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

  - mean μ, standard deviation σ
- For one-dimensional distributions:
  - ~68.3% of all realizations of X are in the interval $[\mu - \sigma, \mu + \sigma]$
  - ~95.4% are in $[\mu - 2\sigma, \mu + 2\sigma]$
  - ~99.7% are in $[\mu - 3\sigma, \mu + 3\sigma]$

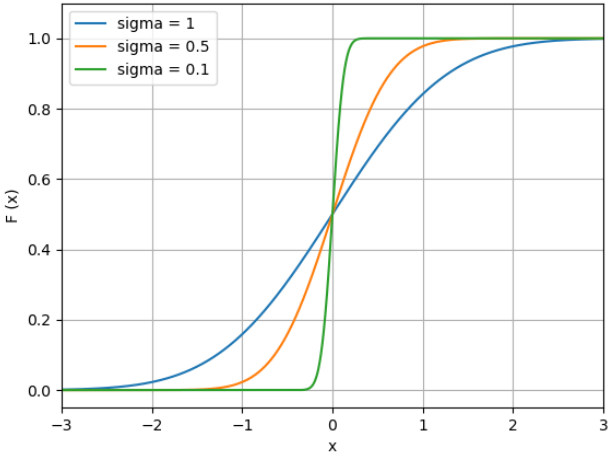
    („3-Sigma Rule“)



Gaussian/Normal Distribution



Probability density function $\mu = 0$



(Cumulative) distribution function, $\mu = 0$

Central Limit Theorem

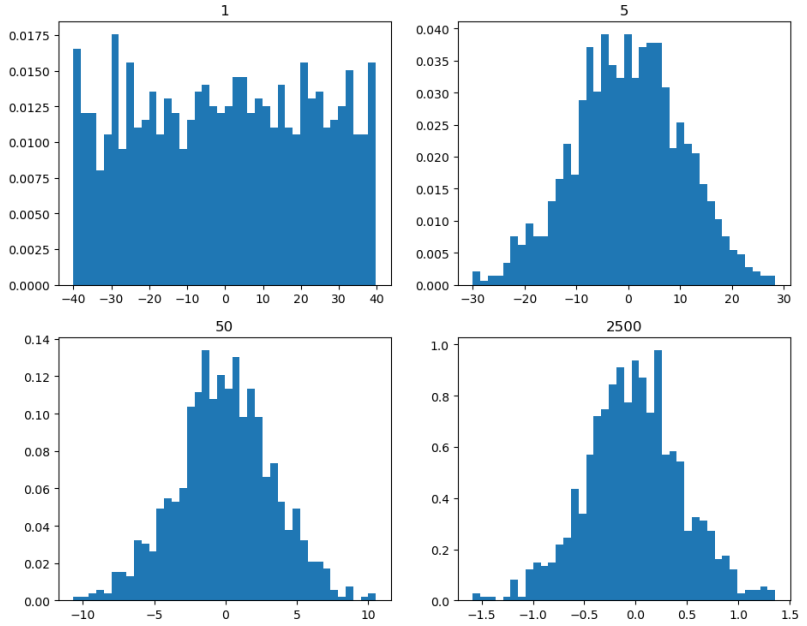
- The average of differently, stochastically independently distributed random variables approaches a Gaussian distribution with an increasing number of samples, i. e.. $X_1, X_2, \ldots, X_n$ are stochastically independent random variables, then

$$\overline{X} = \lim_{N \to \infty} \left(\frac{1}{N} \sum_{i=1}^{N} X_i \right)$$

  is normally distributed.
- This is independent of the underlying distribution of X_i
- The central limit theorem is therefore the argument for the assumption of a Gaussian distribution in many different applications.

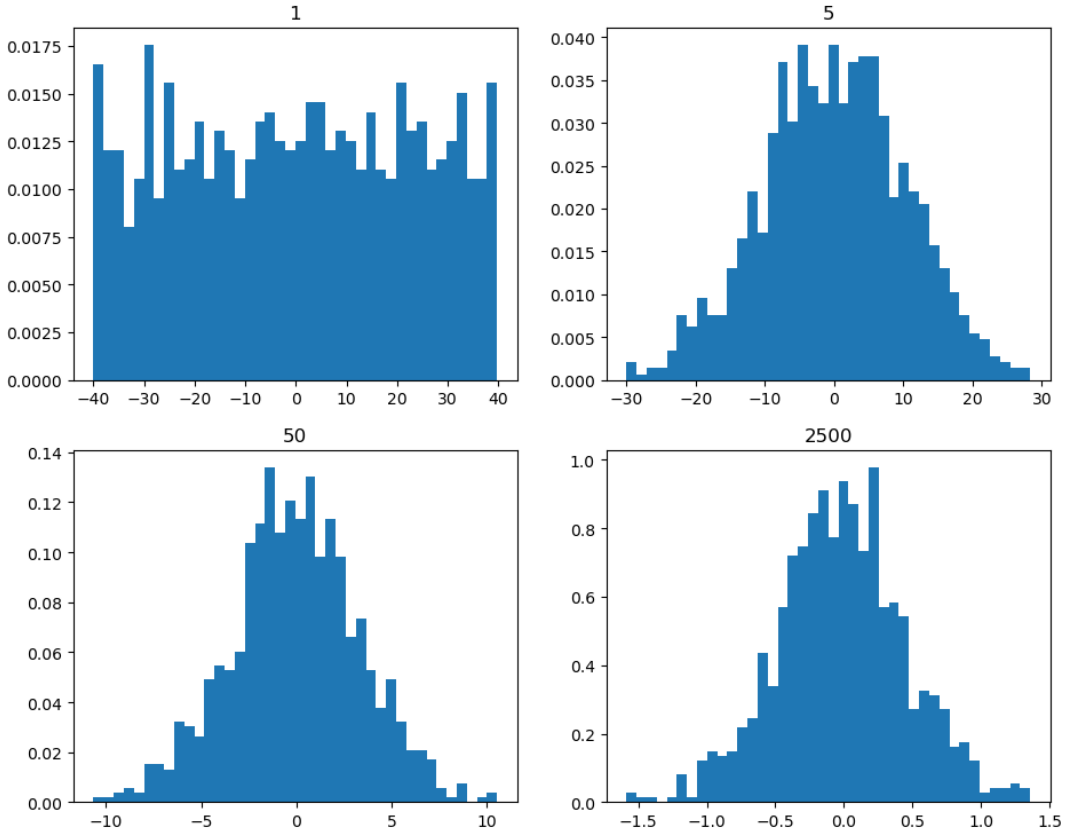
Example:

- Generate 4 sequences of uniformly distributed random variables in the interval $[-40, 40]$ with $N = 1, N = 5$, $N = 50$ and $N = 2500$ and compute the average
- This is done 1000 times and we compute the histogram of the average



Example: Central Limit Theorem

- We generate 4 sequences $X_1, X_2, \ldots X_N$ of uniformly distributed random variables in the interval $[-40, 40]$ with $N = 1, N = 5,$ $N = 50$ and $N = 2500$ and compute the average
- This is done 1000 times and we compute the histogram of the average



Histograms of the avarages of the sequances uniformly distributed random variables

Important Moments of a Probability Distribution

- The i-th **moment** (non-central, non-standardized) of a probability distribution is computed as

$$E\left[X^i\right] = \int_{-\infty}^{\infty} x^i \cdot p_X(x)dx \quad \text{or} \quad E\left[X^i\right] = \sum_{k\in\mathcal{N}} x(k)^i \cdot p_X(x(k))$$

- Example for our dice throw: $E[\text{dice}] = \frac{1}{6} + \frac{2}{6} + \frac{3}{6} + \frac{4}{6} + \frac{5}{6} + \frac{6}{6} = \frac{21}{6} = 3.5$

- First **raw** moment: **mean** μ. $\mu \equiv E[X]$

- **Second central moment: variance** $\text{Var}(X) = E\left[(X - E\left[X\right])^2\right]$

  Its square root is called the **standard deviation** $\sigma = \sqrt{\text{Var}[X]}$

Computational Rules for the Mean

- For two random variables X, Y and constants a, b :

$$\begin{aligned} E[a] &= a, \\ E[aX] &= aE[X], \\ E[a+bX] &= a+bE[X], \\ E[X+Y] &= E[X]+E[Y] \end{aligned}$$

- For stochastically independent random variables X, Y:

$$E[X \cdot Y] = E[X] \cdot E[Y]$$

Computational Rules for the Variance

- For two random variables X, Y and constants a, b :

$$\begin{aligned} \mathrm{Var}[a] &= 0, \\ \mathrm{Var}[a+X] &= \mathrm{Var}[X], \\ \mathrm{Var}[a+bX] &= b^2 \cdot \mathrm{Var}[X], \\ \mathrm{Var}[X+Y] &= \mathrm{Var}[X] + \mathrm{Var}[Y] \\ &\quad + 2 \cdot \underbrace{(E[X \cdot Y] - E[X]\,E[Y])}_{\mathrm{Cov}(X,Y)} \end{aligned}$$

- For statistically independent random variables X and Y:

$$\mathrm{Var}[X+Y] = \mathrm{Var}[X] + \mathrm{Var}[Y]$$

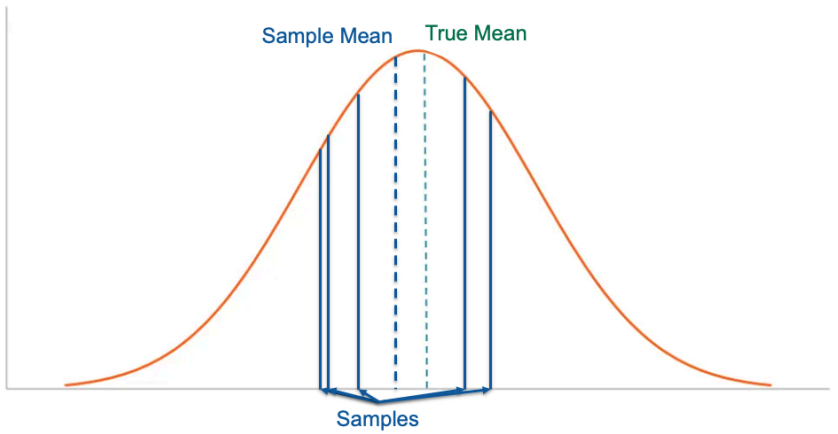
Population and Sample Mean and Variance

- **Our rules so far spoke of *probability density functions* which (in the discrete case) determine the likelihood of each event x_i in the population X.**
- **When we try to determine the mean and variance for the PDF, we**
  - sample N values $x_1, \ldots, x_n$ randomly
  - calculate the mean $\bar{x}$ and variance s^2 as estimates for μ, σ^2 .
  - Knowing nothing about the distribution, we have to assume $p_X(X_i) = 1/N$ for each sample.
- **With $\sigma^2 = E[(X-\mu)^2] = \sum (x_i - \mu)^2 p_X(x_i)$, we could assume $s^2 = \frac{1}{N} \sum (x_i - \bar{x})^2$ for the variance.**
- **However this would result in an estimate biased towards the mean.**
- **The unbiased term is obtained by multiplying with *Bessel's Correction* $N/(N-1)$*:***

$$s^2 = \frac{1}{N-1} \sum (x_i - \bar{x})^2.$$

Bessel‘s Correction

- **Intuitive explanation:**
  - By pulling a sample set, we don‘t calculate the actual population mean μ but the mean of the sample set $\bar{x}$.
  - Therefore on average $\sum(x_i - \bar{x})^2$ can be expected to be smaller than $\sum(x_i - \mu)^2$ - we will be consistently underestimating the variance.
  - Dividing by $N - 1$ instead of N introduces a small correction factor.
- **For large sample sets or the whole population ($N \to \infty$), $N/(N-1)$ converges to 1 and we have the original term.**
- **The true mathematical derivation of Bessel‘s Correction is quite involved.**
- **There are a number of examples on the Internet for self-study, e.g. [1]**



Co-Variance

Covariance $Cov(X, Y)$

- Is a measure for the stochastic dependency between X and Y defined as

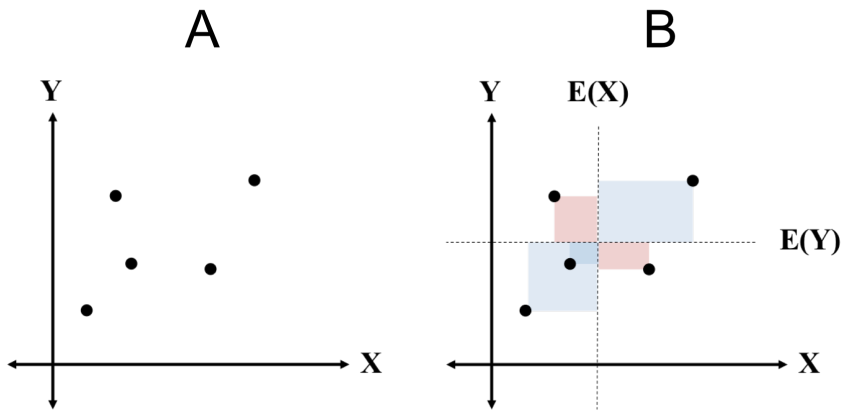
$$\begin{aligned}\mathrm{Cov}(X,Y) &= E\left[(X - E[X])\,(Y - E[Y])\right] \\ &= E[X \cdot Y] - 2 \cdot E[X] \cdot E[Y] + E[X] \cdot E[Y] \\ &= E[X \cdot Y] - E[X] \cdot E[Y].\end{aligned}$$

- Computational rules for the covariance

$$\mathrm{Cov}(X, X) = \mathrm{Var}(X),$$

$$\mathrm{Cov}(a + bX, c + dY) = b \cdot d \cdot \mathrm{Cov}(X, Y),$$

$$\mathrm{Cov}(X + Y, Z) = \mathrm{Cov}(X, Z) + \mathrm{Cov}(Y, Z)$$



Visualization of the covariance

- The **correlation coefficient** (normalized with values between -1 and $+1$) measures the degree of correlation between the two variables:

$$r_{XY} = \frac{\mathrm{Cov}(X, Y)}{\sqrt{\mathrm{Var}[X] \cdot \mathrm{Var}[Y]}}$$

so a high stochastic dependency means $|r_{XY}|$ being close to 1. Independence means $r_{XY} \approx 0$.

- **For generalized multivariate random variables, the expectation (= mean) is**

$$E[\mathbf{x}] = \int_{-\infty}^{\infty} \dots \int_{-\infty}^{\infty} \vec{x} p(\mathbf{x} = \vec{x}) dx_1 \dots dx_n$$

or for the discrete case

$$E[\mathbf{x}] = \sum \sum \cdots \sum_{n} \vec{x} \cdot P(x_1, x_2, \dots, x_n)$$

- **The Covariance, which**
  - for two variables is $\mathrm{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$,
  - becomes the **Covariance Matrix** $\mathrm{Cov}_{\mathbf{xx}} = E[(\mathbf{x} - E(\mathbf{x}))(\mathbf{x} - E(\mathbf{x}))^\top]$.

Covariance

$$\begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \\ \vdots \\ x_n - \mu_n \end{bmatrix} \begin{bmatrix} x_1 - \mu_1 & x_2 - \mu_2 & \dots & x_n - \mu_n \end{bmatrix}$$

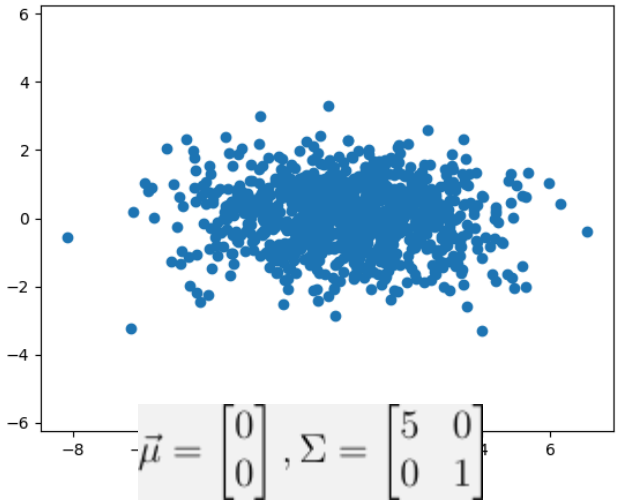
$$= \begin{bmatrix} (x_1 - \mu_1)^2 & (x_1 - \mu_1)(x_2 - \mu_2) & \dots & (x_1 - \mu_1)(x_n - \mu_n) \\ (x_2 - \mu_2)(x_1 - \mu_1) & (x_2 - \mu_2)^2 & \dots & (x_2 - \mu_2)(x_n - \mu_n) \\ \vdots & \vdots & \ddots & \vdots \\ (x_n - \mu_n)(x_1 - \mu_1) & (x_n - \mu_n)(x_2 - \mu_2) & \dots & (x_n - \mu_n)^2 \end{bmatrix}$$

$$= \begin{bmatrix} \sigma^2_{x_1} & \sigma^2_{x_1,x_2} & \dots & \sigma^2_{x_1,x_n} \\ \sigma^2_{x_2,x_1} & \sigma^2_{x_2} & \dots & \sigma^2_{x_2,x_n} \\ \dots & \dots & \ddots & \vdots \\ \sigma^2_{x_n,x_1} & \sigma^2_{x_n,x_2} & \dots & \sigma^2_{x_n} \end{bmatrix}$$

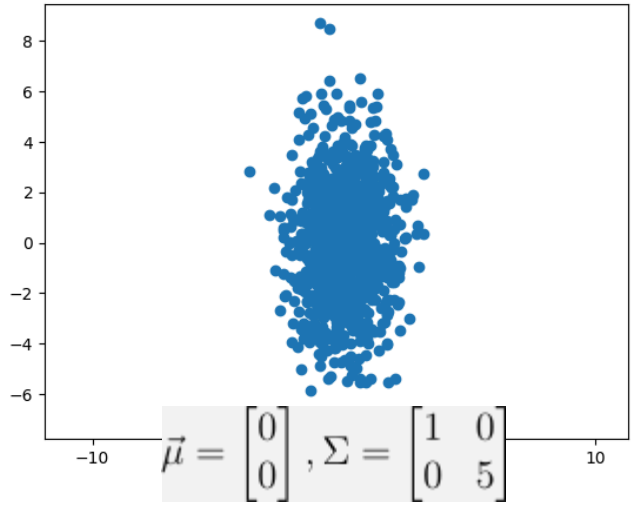
Some properties of the Covariance matrix...

- Always square.
- Diagonal elements are the variances of the vector elements.
- Elements to the side of the diagonal are the co-variances.
- Symmetric (i.e. $\mathrm{Cov}^\top = \mathrm{Cov}$).
- Positive semi-definite (i.e. $\mathbf{x}\mathrm{Cov}\mathbf{x}^\top \geq 0 \forall \mathbf{x} \neq \mathbf{0}$ -- the real-valued product of any nonzero vector with the matrix and the vector's transpose is larger or equal zero).
- Positive definite and invertible if all x_i have variance != 0.

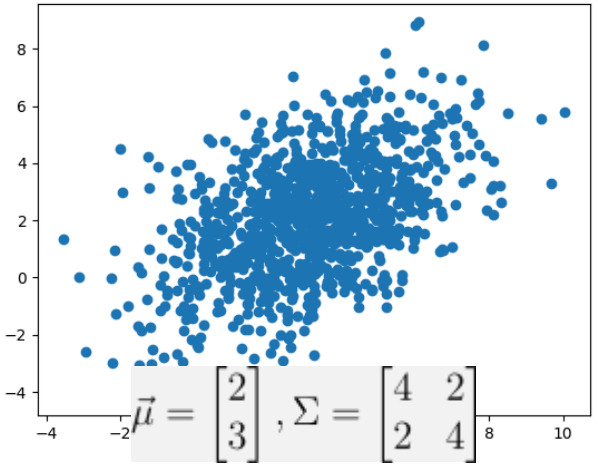
Scatter Plots of Normal Distributions with different means and covariances



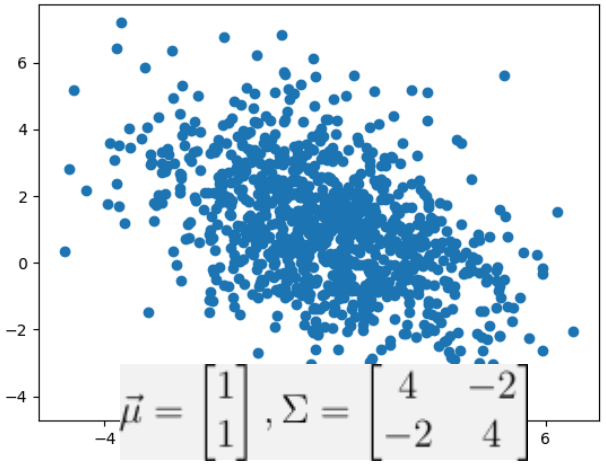
$$\vec{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix}$$



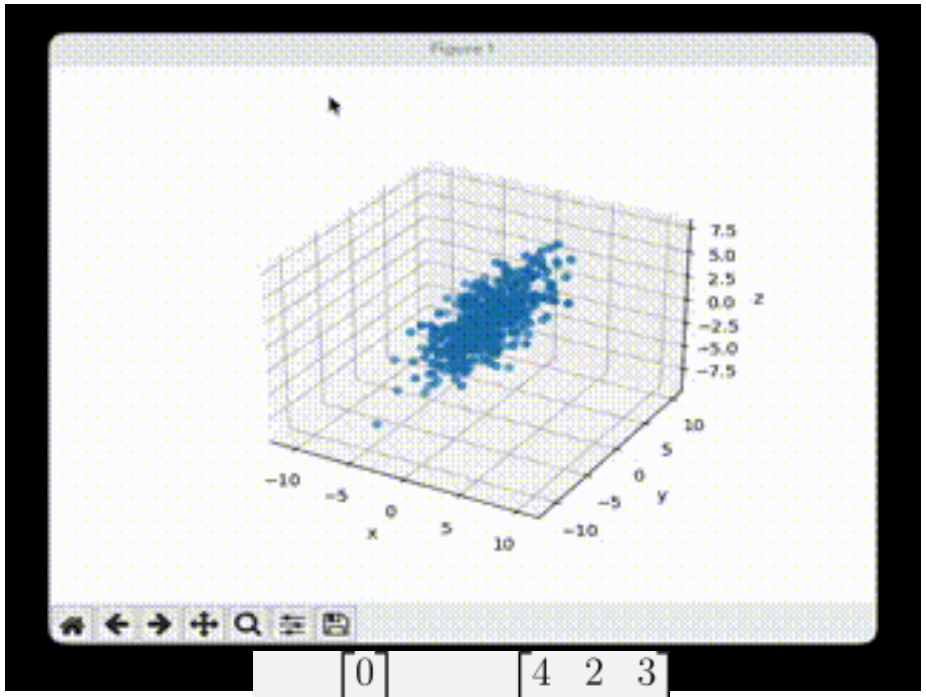
$$\vec{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 5 \end{bmatrix}$$



$$\vec{\mu} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \Sigma = \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix}$$



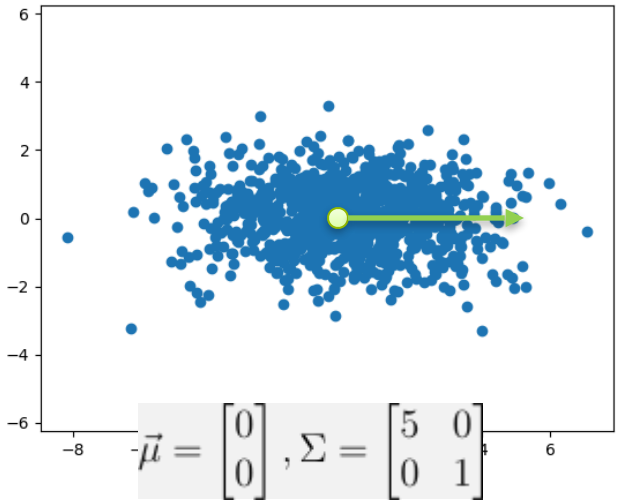
$$\vec{\mu} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \Sigma = \begin{bmatrix} 4 & -2 \\ -2 & 4 \end{bmatrix}$$

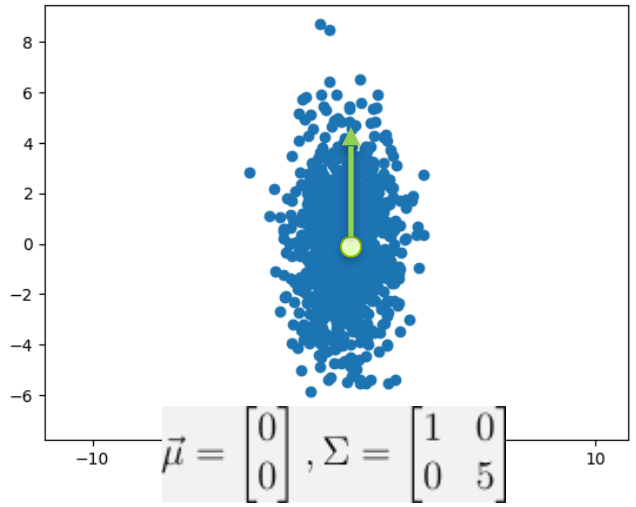


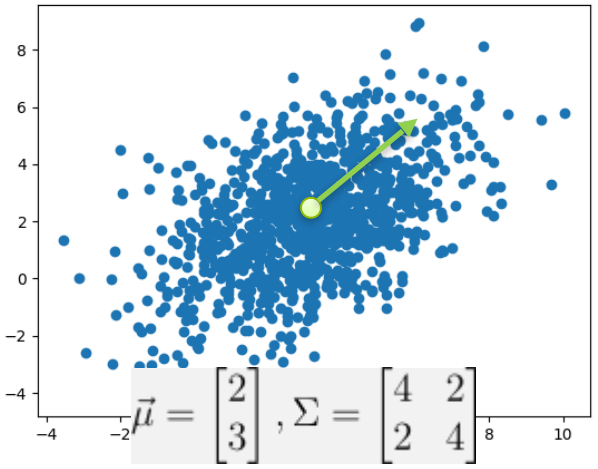
$$\vec{\mu} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 4 & 2 & 3 \\ 2 & 5 & 3 \\ 3 & 3 & 6 \end{bmatrix}$$

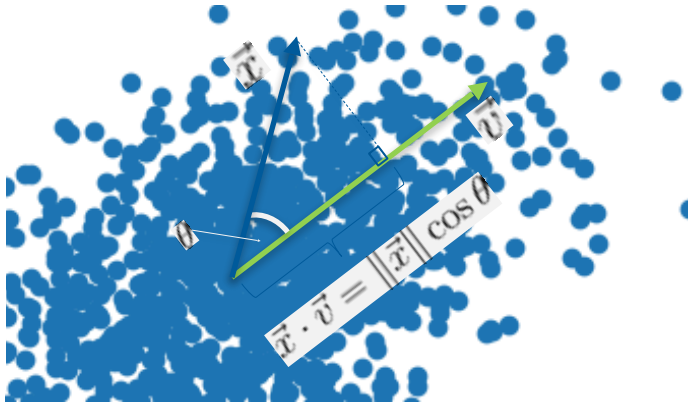
16.01.26

Co-Variance and Eigenvalues









- The covariance matrix ($\sum$ from now on) defines spread (variance) and orientation (covariance) of our data.
- If we want to represent the covariance matrix with a vector and its magnitude, we should try to find the vector that points into the direction of the largest spread of the data.
- The spread (variance) in this direction can give us the magnitude.
- Let‘s call that vector $\vec{v}$.
- We can project all of our data onto that vector with the dot product $\vec{v}^\top \vec{x} = \|\vec{x}\| \cos\theta$.
- The variance of the projected data is $\vec{v}^\top \Sigma \vec{v}$.
- We want to find the vector $\vec{v}$ that maximizes $\vec{v}^\top \Sigma \vec{v}$.

- **Maximizing any function of the form $\vec{v}^\top \Sigma \vec{v}$ with respect to $\vec{v}$ can be done with the so called *Rayleigh Quotient:***

$$R(M, \vec{x}) = \frac{\vec{x}^\top M \vec{x}}{\vec{x}^\top \vec{x}}$$

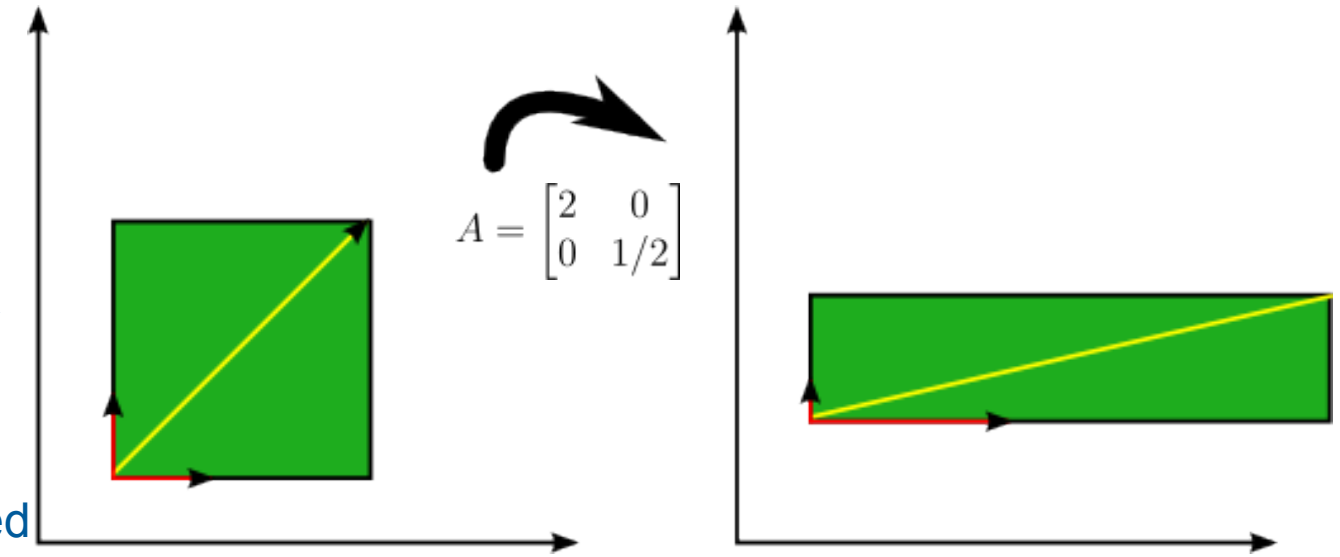
- **It can be shown that $R(M, \vec{x})$ reaches its maximum value $\lambda_{\max}$ when $\vec{x} = \vec{v}_{\max}$, the largest eigenvalue of M.**
- **The largest eigenvector of the covariance matrix always points in the direction of the largest variance,**
  and its magnitude equals the corresponding eigenvalue.
- **The second largest eigenvector is always orthogonal to the largest eigenvector, and points into the direction of the second largest spread of data.**
- ...and so on for all *n* dimensions of the matrix.

Revisiting Eigenvectors and Eigenvalues

- *Eigenvectors* / *eigenvalues* have many important applications in machine learning and computer vision.
- An eigenvector is a vector whose direction remains unchanged when a linear transformation is applied.
- Consider the scaling transform $\vec{v}' = A\vec{v}$ on the right. The yellow vector changes direction, the red vectors do not.
- Therefore the red vectors are called eigenvectors of A.
- Generally, the eigenvectors $\vec{v}_i$ of a matrix A are vectors that satisfy

$$A\vec{v}_i = \lambda_i \vec{v}_i$$

where λ_i are corresponding scalars called eigenvalues.



Revisiting Eigenvectors and Eigenvalues

- We can rewrite the eigenvector equation from the next slide as follows:

$$A\vec{v}_i = \lambda_i \vec{v}_i$$
$$A\vec{v}_i - \lambda_i \vec{v}_i = 0$$
$$\vec{v}_i(A - \lambda_i I) = 0$$

  where I is the identity matrix of the same dimensions as A.

- This is of course true for the trivial solution of $\vec{v}_i = \vec{0}$.
- If $\vec{v}_i$ is not the null vector, $\vec{v}_i(A - \lambda_i I) = 0$ can only be defined if $(A - \lambda_i I)$ is not invertible.
- For a square matrix not to be invertible, its determinant must be zero.
- Therefore, we can find λ_i by solving $\mathrm{Det}(A - \lambda I) = 0$.

Eigenvectors and Inverse Matrices

- Eigenvector of a matrix $\boldsymbol{A}$ corresponding to eigenvalue λ_i given by

$$(\boldsymbol{A} - \lambda_i \boldsymbol{I})\, \boldsymbol{v}_i = \boldsymbol{0} \quad \text{or} \quad \boldsymbol{A}\boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i$$

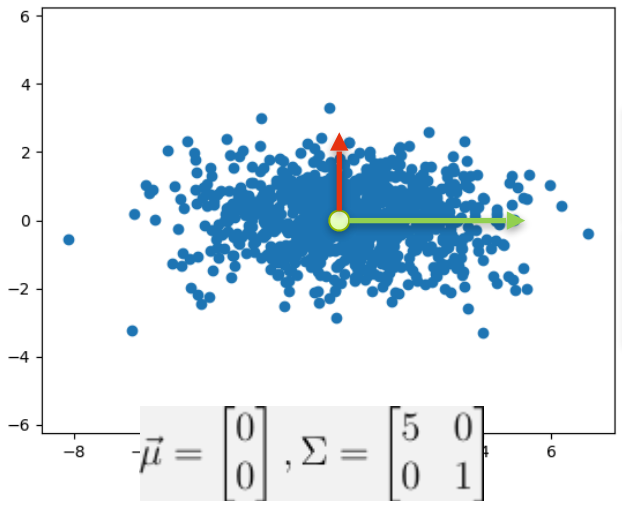
- Multiplying from the left with $\boldsymbol{A}^{-1}$ we obtain

$$\boldsymbol{v}_i = \lambda_i \boldsymbol{A}^{-1} \boldsymbol{v}_i$$

$$\leftrightarrow \boldsymbol{A}^{-1} \boldsymbol{v}_i = \frac{1}{\lambda_i} \boldsymbol{v}_i$$

- So, the eigenvector $\boldsymbol{v}_i$ of $\boldsymbol{A}$ is also an eigenvector of $\boldsymbol{A}^{-1}$ for the eigenvalue $1/\lambda_i$

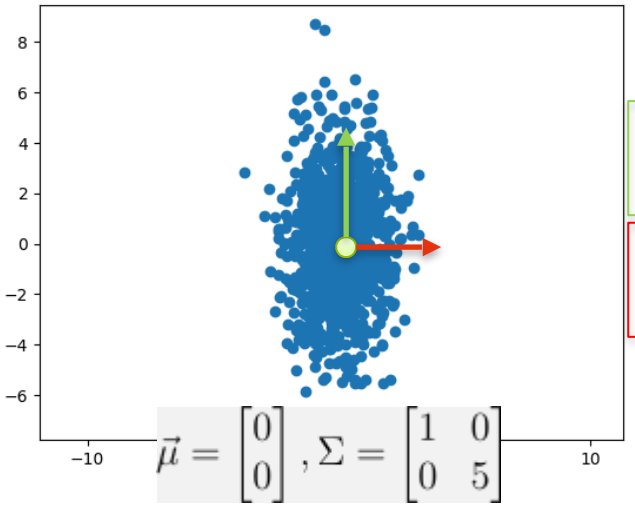
Some examples:



$$\vec{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\|v_1\| = \lambda_1 = 5, v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

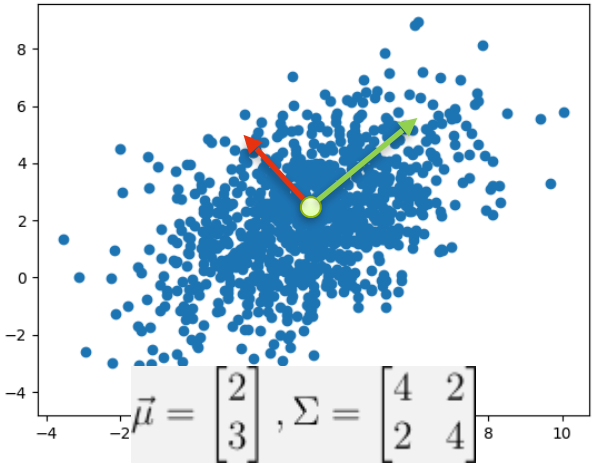
$$\|v_2\| = \lambda_2 = 1, v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$



$$\vec{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 5 \end{bmatrix}$$

$$\|v_1\| = \lambda_1 = 5, v_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\|v_2\| = \lambda_2 = 1, v_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

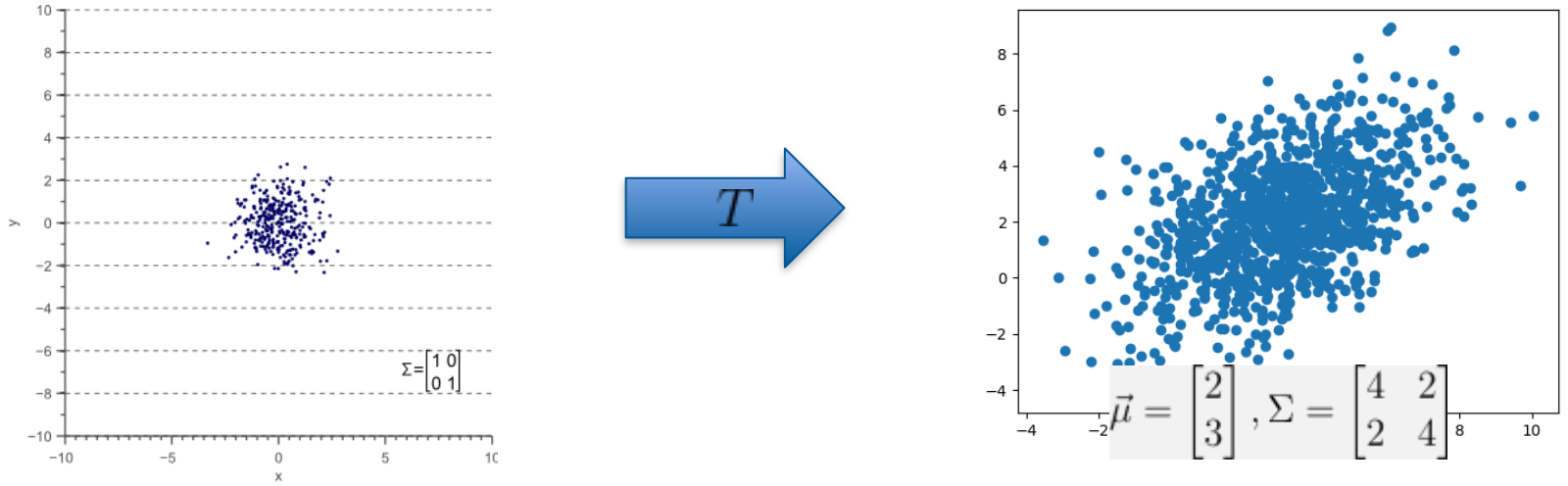


$$\vec{\mu} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \Sigma = \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix}$$

$$\|v_1\| = \lambda_1 = 6, v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\|v_2\| = \lambda_2 = 2, v_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Covariance Matrix as a Linear Transformation



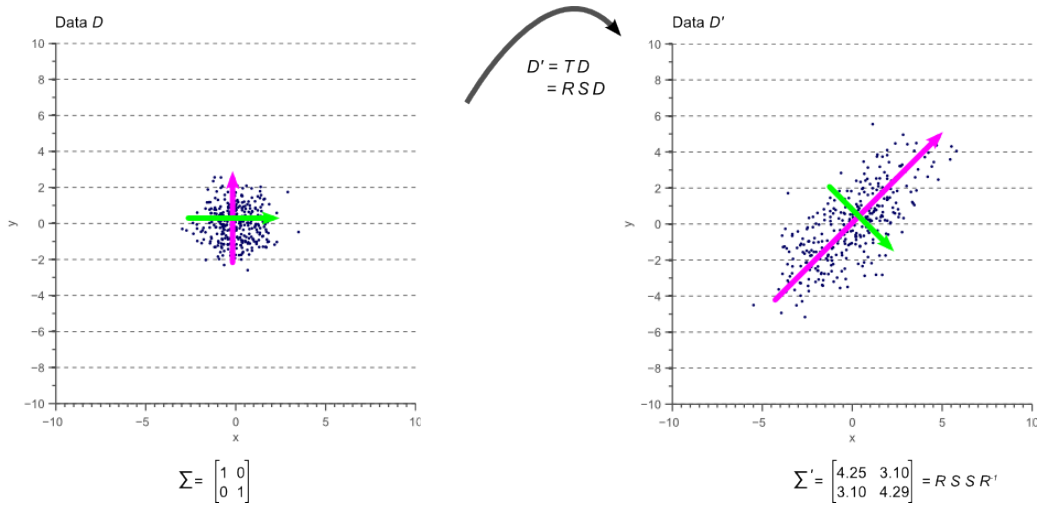
D $\quad D' = TD + \vec{\mu}$ $\quad D'$

where T is the product of a rotation matrix R and a scaling matrix S:

$$T = RS$$

$\vec{\mu}$ is a translation vector representing the mean.

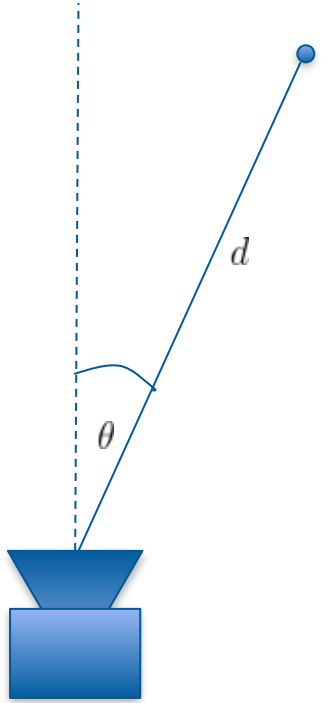
- **If we have**
  - **an original data set D with a covariance matrix Σ, and**
  - **a linear transformation T given by a rotation R, translation (mean shift) $\vec{\mu}$, and scaling matrix S, such that $D' = TD + \vec{\mu} = RSD + \vec{\mu}$,**
- **the covariance matrix of the transformed data set D' is calculated by $\Sigma' = TT^{\top}\Sigma = RS(SR)^{\top}\Sigma$.**



Putting it all together – Sensor Modelling Example

- **Let‘s assume we have a sensor that delivers distance over azimuth.**
- **We take 1000 measurements at various angles and distances and find:**
  - all measurements within $\pm 3°$ across the whole field of view
  - all measurements within $\pm 1\%$ depending on distance.

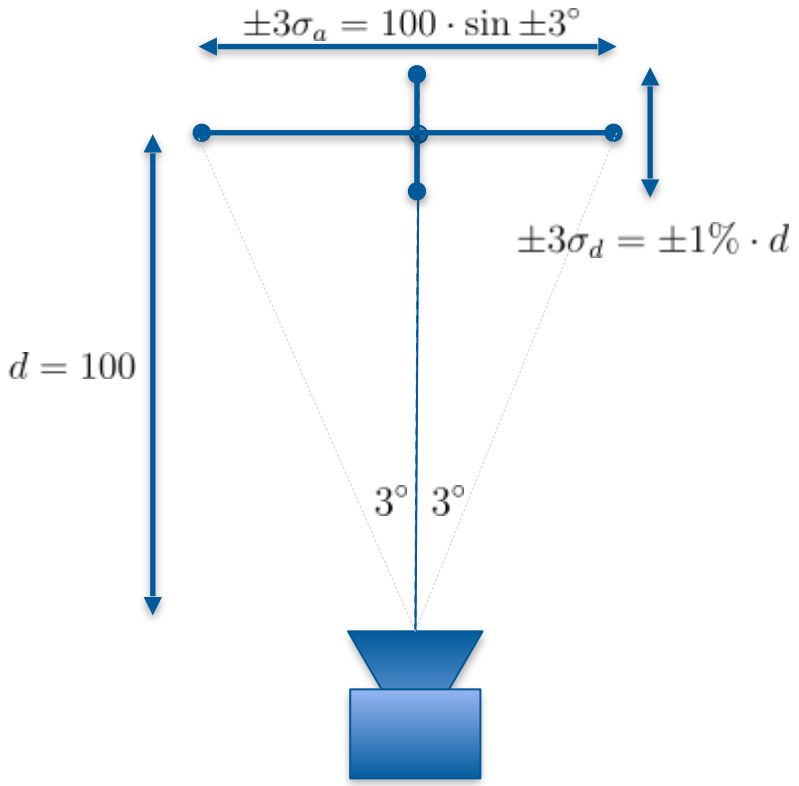
- **What is the covariance matrix of an object measured at 20°of azimuth, 100m of distance?**



- **We start with an unrotated normal distribution of** $\Sigma_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$.
- **Since we empirically measured all measurements within** $\pm 1\%$ **of distance, we can assume** $3\sigma_d = 100 \cdot 1\%$

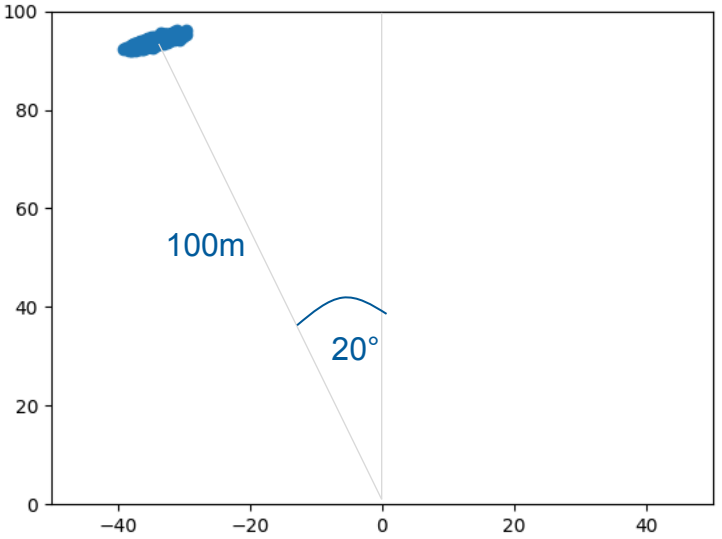
  or $\sigma_d = \frac{1}{3}$ **meters.**
- **Since we empirically measured all measurements with** $\pm 3^\circ$ **of azimuth, we can assume** $3\sigma_a = 100 \cdot \sin 3^\circ$

  or $\sigma_a \approx \frac{5}{3}$ **meters.**
- **So our unrotated data is scaled by** $\begin{bmatrix} \frac{5}{3} & 0 \\ 0 & \frac{1}{3} \end{bmatrix}$.



- **We see the object at 20° of azimuth, so we need to rotate by**

$$\begin{bmatrix} \cos 20° & -\sin 20° \\ \sin 20° & \cos 20° \end{bmatrix} \approx \begin{bmatrix} 0.94 & -0.342 \\ 0.342 & 0.94 \end{bmatrix}$$



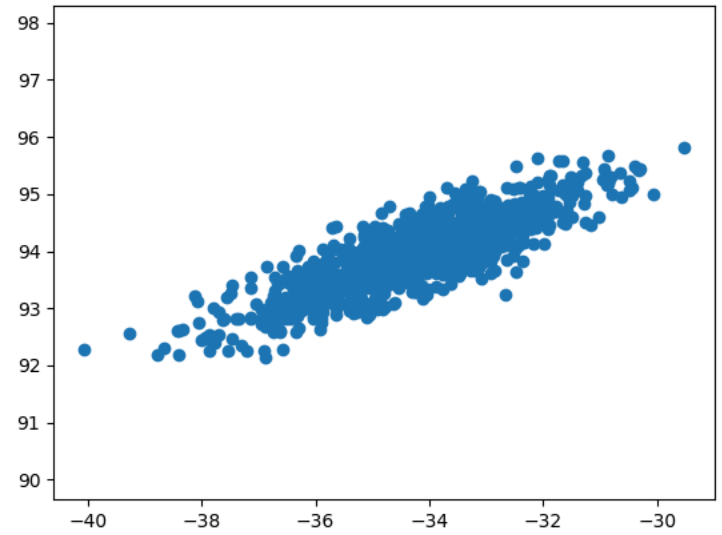
- **For a data transform of** $T = RS = \begin{bmatrix} \cos 20° & -\sin 20° \\ \sin 20° & \cos 20° \end{bmatrix} \begin{bmatrix} \frac{5}{3} & 0 \\ 0 & \frac{1}{3} \end{bmatrix}$

we obtain a covariance matrix of

$$\Sigma' = RS(SR)^T = \begin{bmatrix} \cos 20° & -\sin 20° \\ \sin 20° & \cos 20° \end{bmatrix} \begin{bmatrix} \frac{5}{3} & 0 \\ 0 & \frac{1}{3} \end{bmatrix} \begin{bmatrix} \frac{5}{3} & 0 \\ 0 & \frac{1}{3} \end{bmatrix} \begin{bmatrix} \cos 20° & \sin 20° \\ -\sin 20° & \cos 20° \end{bmatrix}$$

and can use this to generate a scatter plot of the multivariate normal distribution at

$$\vec{\mu} = \begin{bmatrix} 100 \sin \theta \\ 100 \cos \theta \end{bmatrix} \approx \begin{bmatrix} 34.2 \\ 93.97 \end{bmatrix}.$$





Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!